PROGRAMMING LANGUAGE PRAGMATICS

FOURTH EDITION

Michael L. Scott

MORGAN KAUFMANN

Programming Language Pragmatics

FOURTH EDITION

7 Type Systems

Most programming languages include a notion of *type* for expressions and/or objects.[1] Types serve several important purposes:

EXAMPLE 7.1
Operations that leverage type information

1. Types provide implicit context for many operations, so that the programmer does not have to specify that context explicitly. In C, for instance, the expression `a + b` will use integer addition if `a` and `b` are of integer (`int`) type; it will use floating-point addition if `a` and `b` are of floating-point (`double` or `float`) type. Similarly, the operation `new p` in Pascal, where `p` is a pointer, will allocate a block of storage from the heap that is the right size to hold an object of the type pointed to by `p`; the programmer does not have to specify (or even know) this size. In C++, Java, and C#, the operation `new my_type()` not only allocates (and returns a pointer to) a block of storage sized for an object of type `my_type`; it also automatically calls any user-defined initialization (*constructor*) function that has been associated with that type. ■

EXAMPLE 7.2
Errors captured by type information

2. Types limit the set of operations that may be performed in a semantically valid program. They prevent the programmer from adding a character and a record, for example, or from taking the arctangent of a set, or passing a file as a parameter to a subroutine that expects an integer. While no type system can promise to catch every nonsensical operation that a programmer might put into a program by mistake, good type systems catch enough mistakes to be highly valuable in practice. ■
3. If types are specified explicitly in the source program (as they are in many but not all languages), they can often make the program easier to read and understand. In effect, they serve as stylized documentation, whose correctness is checked by the compiler. (On the flip side, the need for this documentation can sometimes make the program harder to write.)
4. If types are known at compile time (either because the programmer specifies them explicitly or because the compiler is able to infer them), they can be used

[1] Recall that unless otherwise noted we are using the term "object" informally to refer to anything that might have a name. Object-oriented languages, which we will study in Chapter 10, assign a narrower, more formal, meaning to the term.

EXAMPLE 7.3
Types as a source of "may alias" information

to drive important performance optimizations. As a simple example, recall the concept of *aliases*, introduced in Section 3.5.1, and discussed in Sidebar 3.7. If a program performs an assignment through a pointer, the compiler may be able to infer that objects of unrelated types cannot possibly be affected; their values can safely remain in registers, even if loaded prior to the assignment. ■

Section 7.1 looks more closely at the meaning and purpose of types. It presents some basic definitions, and introduces the notions of polymorphism and orthogonality. Section 7.2 takes a closer look at type checking; in particular, it considers *type equivalence* (when can we say that two types are the same?), *type compatibility* (when can we use a value of a given type in a given context?), and *type inference* (how do we deduce the type of an expression from the types of its components and that of the surrounding context?).

As an example of both polymorphism and sophisticated inference, Section 7.2.4 surveys the type system of ML, which combines, to a large extent, the efficiency and early error reporting of compilation with the convenience and flexibility of interpretation. We continue the study of polymorphism in Section 7.3, with a particular emphasis on *generics*, which allow a body of code to be parameterized explicitly for multiple types. Finally, in Section 7.4, we consider what it means to compare two complex objects for equality, or to assign one into the other. In Chapter 8 we will consider syntactic, semantic, and pragmatic issues for some of the most important *composite* types: records, arrays, strings, sets, pointers, lists, and files.

7.1 Overview

Computer hardware can interpret bits in memory in several different ways: as instructions, addresses, characters, and integer and floating-point numbers of various lengths. The bits themselves, however, are untyped: the hardware on most machines makes no attempt to keep track of which interpretations correspond to which locations in memory. Assembly languages reflect this lack of typing: operations of any kind can be applied to values in arbitrary locations. High-level languages, by contrast, almost always associate types with values, to provide the contextual information and error checking alluded to above.

Informally, a *type system* consists of (1) a mechanism to define types and associate them with certain language constructs, and (2) a set of rules for *type equivalence*, *type compatibility*, and *type inference*. The constructs that must have types are precisely those that have values, or that can refer to objects that have values. These constructs include named constants, variables, record fields, parameters, and sometimes subroutines; literal constants (e.g., `17`, `3.14`, `"foo"`); and more complicated expressions containing these. Type equivalence rules determine when the types of two values are the same. Type compatibility rules determine when a value of a given type can be used in a given context. Type inference rules define the type of an expression based on the types of its constituent parts or

(sometimes) the surrounding context. In a language with polymorphic variables or parameters, it may be important to distinguish between the type of a reference or pointer and the type of the object to which it refers: a given name may refer to objects of different types at different times.

Subroutines are considered to have types in some languages, but not in others. Subroutines need to have types if they are first- or second-class values (i.e., if they can be passed as parameters, returned by functions, or stored in variables). In each of these cases there is a construct in the language whose value is a dynamically determined subroutine; type information allows the language to limit the set of acceptable values to those that provide a particular subroutine interface (i.e., particular numbers and types of parameters). In a statically scoped language that never creates references to subroutines dynamically (one in which subroutines are always third-class values), the compiler can always identify the subroutine to which a name refers, and can ensure that the routine is called correctly without necessarily employing a formal notion of subroutine types.

Type checking is the process of ensuring that a program obeys the language's type compatibility rules. A violation of the rules is known as a *type clash*. A language is said to be *strongly typed* if it prohibits, in a way that the language implementation can enforce, the application of any operation to any object that is not intended to support that operation. A language is said to be *statically typed* if it is strongly typed and type checking can be performed at compile time. In the strictest sense of the term, few languages are statically typed. In practice, the term is often applied to languages in which most type checking can be performed at compile time, and the rest can be performed at run time.

Since the mid 1970s, most newly developed languages have tended to be strongly (though not necessarily statically) typed. Interestingly, C has become more strongly typed with each successive version of the language, though various loopholes remain; these include unions, nonconverting type casts, subroutines with variable numbers of parameters, and the interoperability of pointers and arrays (to be discussed in Section 8.5.1). Implementations of C rarely check anything at run time.

DESIGN & IMPLEMENTATION

7.1 Systems programming

The standard argument against complete type safety in C is that systems programs need to be able to "break" types on occasion. Consider, for example, the code that implements dynamic memory management (e.g., `malloc` and `free`). This code must interpret the same bytes, at different times, as unallocated space, metadata, or (parts of) user-defined data structures. "By fiat" conversions between types are inescapable. Such conversions need not, however, be subtle. Largely in reaction to experience with C, the designers of C# chose to permit operations that break the type system only within blocks of code that have been explicitly labeled `unsafe`.

Dynamic (run-time) type checking can be seen as a form of late binding, and tends to be found in languages that delay other issues until run time as well. Static typing is thus the norm in languages intended for performance; dynamic typing is more common in languages intended for ease of programming. Lisp and Smalltalk are dynamically (though strongly) typed. Most scripting languages are also dynamically typed; some (e.g., Python and Ruby) are strongly typed. Languages with dynamic scoping are generally dynamically typed (or not typed at all): if the compiler can't identify the object to which a name refers, it usually can't determine the type of the object either.

7.1.1 The Meaning of "Type"

While every programmer has at least an informal notion of what is meant by "type," that notion can be formalized in several different ways. Three of the most popular are what we might call the *denotational*, *structural*, and *abstraction-based* points of view. From the denotational point of view, a type is simply a set of values. A value has a given type if it belongs to the set; an object has a given type if its value is guaranteed to be in the set. From the structural point of view, a type is either one of a small collection of *built-in* types (integer, character, Boolean, real, etc.; also called *primitive* or *predefined* types), or a *composite* type created by

DESIGN & IMPLEMENTATION

7.2 Dynamic typing

The growing popularity of scripting languages has led a number of prominent software developers to publicly question the value of static typing. They ask: given that we can't check everything at compile time, how much pain is it worth to check the things we can? As a general rule, it is easier to write type-correct code than to prove that we have done so, and static typing requires such proofs. As type systems become more complex (due to object orientation, generics, etc.), the complexity of static typing increases correspondingly.

Anyone who has written extensively in Ada or C++ on the one hand, and in Python or Scheme on the other, cannot help but be struck at how much easier it is to write code, at least for modest-sized programs, without complex type declarations. Dynamic checking incurs some run-time overhead, of course, and may delay the discovery of bugs, but this is increasingly seen as insignificant in comparison to the potential increase in human productivity. An intermediate position, epitomized by the ML family of languages but increasingly adopted (in limited form) by others, retains the requirement that types be statically known, but relies on the compiler to infer them automatically, without the need for some (or—in the case of ML—most) explicit declarations. We will discuss this topic more in Section 7.2.3. Static and dynamic typing and the role of inference promise to provide some of the most interesting language debates of the coming decade.

applying a type *constructor* (`record`, `array`, `set`, etc.) to one or more simpler types. (This use of the term "constructor" is unrelated to the initialization functions of object-oriented languages. It also differs in a more subtle way from the use of the term in ML.) From the abstraction-based point of view, a type is an *interface* consisting of a set of operations with well-defined and mutually consistent semantics. For both programmers and language designers, types may also reflect a mixture of these viewpoints.

In denotational semantics (one of several ways to formalize the meaning of programs), a set of values is known as a *domain*. Types are domains, and the meaning of an expression is a value from the domain that represents the expression's type. Some domains—the integers, for example—are simple and familiar. Others are more complex. An array can be thought of as a value from a domain whose elements are functions; each of these functions maps values from some finite *index* type (typically a subset of the integers) to values of some other *element* type. As it turns out, denotational semantics can associate a type with *everything* in a program—even statements with side effects. The meaning of an assignment statement is a value from a domain of higher-level functions, each of whose elements maps a *store*—a mapping from names to values that represents the current contents of memory—to another store, which represents the contents of memory after the assignment.

One of the nice things about the denotational view of types is that it allows us in many cases to describe user-defined composite types (records, arrays, etc.) in terms of mathematical operations on sets. We will allude to these operations again under "Composite Types" in Section 7.1.4. Because it is based on mathematical objects, the denotational view of types usually ignores such implementation issues as limited precision and word length. This limitation is less serious than it might at first appear: Checks for such errors as arithmetic overflow are usually implemented outside of the type system of a language anyway. They result in a run-time error, but this error is not called a type clash.

When a programmer defines an enumerated type (e.g., `enum hue {red, green, blue}` in C), he or she certainly thinks of this type as a set of values. For other varieties of user-defined type, this denotational view may not be as natural. Instead, the programmer may think in terms of the way the type is built from simpler types, or in terms of its meaning or purpose. These ways of thinking reflect the structural and abstraction-based points of view, respectively. The structural point of view was pioneered by Algol W and Algol 68, and is characteristic of many languages designed in the 1970s and 1980s. The abstraction-based point of view was pioneered by Simula-67 and Smalltalk, and is characteristic of modern object-oriented languages; it can also be found in the module constructs of various other languages, and it can be adopted as a matter of programming discipline in almost any language. We will consider the structural point of view in more detail in Chapter 8, and the abstraction-based in Chapter 10.

7.1.2 Polymorphism

Polymorphism, which we mentioned briefly in Section 3.5.2, takes its name from the Greek, and means "having multiple forms." It applies to code—both data structures and subroutines—that is designed to work with values of multiple types. To maintain correctness, the types must generally have certain characteristics in common, and the code must not depend on any other characteristics. The commonality is usually captured in one of two main ways. In *parametric polymorphism* the code takes a type (or set of types) as a parameter, either explicitly or implicitly. In *subtype polymorphism*, the code is designed to work with values of some specific type *T*, but the programmer can define additional types to be extensions or refinements of *T*, and the code will work with these *subtypes* as well.

Explicit parametric polymorphism, also known as *generics* (or *templates* in C++), typically appears in statically typed languages, and is usually implemented at compile time. The implicit version can also be implemented at compile time—specifically, in ML-family languages; more commonly, it is paired with dynamic typing, and the checking occurs at run time.

Subtype polymorphism appears primarily in object-oriented languages. With static typing, most of the work required to deal with multiple types can be performed at compile time: the principal run-time cost is an extra level of indirection on method invocations. Most languages that envision such an implementation, including C++, Eiffel, OCaml, Java, and C#, provide a separate mechanism for generics, also checked mainly at compile time. The combination of subtype and parametric polymorphism is particularly useful for *container* (*collection*) classes such as "list of `T`" (`List<T>`) or "stack of `T`" (`Stack<T>`), where `T` is initially unspecified, and can be instantiated later as almost any type.

By contrast, dynamically typed object-oriented languages, including Smalltalk, Python, and Ruby, generally use a single mechanism for both parametric and subtype polymorphism, with checking delayed until run time. A unified mechanism also appears in Objective-C, which provides dynamically typed objects on top of otherwise static typing.

We will consider parametric polymorphism in more detail in Section 7.3, after our coverage of typing in ML. Subtype polymorphism will largely be deferred to Chapter 10, which covers object orientation, and to Section 14.4.4, which focuses on objects in scripting languages.

7.1.3 Orthogonality

In Section 6.1.2 we discussed the importance of orthogonality in the design of expressions, statements, and control-flow constructs. In a highly orthogonal language, these features can be used, with consistent behavior, in almost any combination. Orthogonality is equally important in type system design. A highly orthogonal language tends to be easier to understand, to use, and to reason about in

a formal way. We have noted that languages like Algol 68 and C enhance orthogonality by eliminating (or at least blurring) the distinction between statements and expressions. To characterize a statement that is executed for its side effect(s), and that has no useful values, some languages provide a trivial type with a single value. In C and Algol 68, for example, a subroutine that is meant to be used as a procedure is generally declared with a return type of `void`. In ML, the trivial type is called `unit`. If the programmer wishes to call a subroutine that does return a value, but the value is not needed in this particular case (all that matters is the side effect[s]), then the return value in C can be discarded by "casting" it to `void`:

EXAMPLE 7.4
`void` (trivial) type

```
foo_index = insert_in_symbol_table(foo);
...
(void) insert_in_symbol_table(bar);     /* don't care where it went */
    /* cast is optional; implied if omitted */
```

In a language (e.g., Pascal) without a trivial type, the latter of these two calls would need to use a dummy variable:

EXAMPLE 7.5
Making do without `void`

```
var dummy : symbol_table_index;
...
dummy := insert_in_symbol_table(bar);
```

As another example of orthogonality, consider the common need to "erase" the value of a variable—to indicate that it does not hold a valid value of its type. For pointer types, we can often use the value `null`. For enumerations, we can add an extra "none of the above" alternative to the set of possible values. But these two techniques are very different, and they don't generalize to types that already make use of all available bit patterns in the underlying implementation.

EXAMPLE 7.6
Option types in OCaml

To address the need for "none of the above" in a more orthogonal way, many functional languages—and some imperative languages as well—provide a special type constructor, often called `Option` or `Maybe`. In OCaml, we can write

```
let divide n d : float option =  (* n and d are parameters *)
  match d with                   (* "float option" is the return type *)
  | 0. -> None
  | _  -> Some (n /. d);;        (* underscore means "anything else" *)

let show v : string =
  match v with
  | None   -> "??"
  | Some x -> string_of_float x;;
```

Here function `divide` returns `None` if asked to divide by zero; otherwise it returns `Some` *x*, where *x* is the desired quotient. Function `show` returns either `"??"` or the string representation of *x*, depending on whether parameter `v` is `None` or `Some` *x*.

EXAMPLE 7.7
Option types in Swift

Option types appear in a variety of other languages, including Haskell (which calls them `Maybe`), Scala, C#, Swift, and (as generic library classes) Java and C++. In the interest of brevity, C# and Swift use a trailing question mark instead of the `option` constructor. Here is the previous example, rewritten in Swift:

```
func divide(n : Double, d : Double) -> Double? {
    if d == 0 { return nil }
    return n / d
}

func show(v : Double?) -> String {
    if v == nil { return "??" }
    return "\(v!)"              // interpolate v into string
}
```

With these definitions, `show(divide(3.0, 4.0))` will evaluate to `"0.75"`, while `show(divide(3.0, 0.0))` will evaluate to `"??"`. ■

Yet another example of orthogonality arises when specifying literal values for objects of composite type. Such literals are sometimes known as *aggregates*. They are particularly valuable for the initialization of static data structures; without them, a program may need to waste time performing initialization at run time.

EXAMPLE 7.8
Aggregates in Ada

Ada provides aggregates for all its structured types. Given the following declarations

```
type person is record
         name : string (1..10);
         age : integer;
     end record;
p, q : person;
A, B : array (1..10) of integer;
```

we can write the following assignments:

```
p := ("Jane Doe  ", 37);
q := (age => 36, name => "John Doe  ");
A := (1, 0, 3, 0, 3, 0, 3, 0, 0, 0);
B := (1 => 1, 3 | 5 | 7 => 3, others => 0);
```

Here the aggregates assigned into `p` and `A` are *positional*; the aggregates assigned into `q` and `B` name their elements explicitly. The aggregate for `B` uses a shorthand notation to assign the same value (`3`) into array elements `3`, `5`, and `7`, and to assign a `0` into all unnamed fields. Several languages, including C, C++, Fortran 90, and Lisp, provide similar capabilities. ■

ML provides a very general facility for composite expressions, based on the use of *constructors* (discussed in Section 11.4.3). Lambda expressions, which we saw in Section 3.6.4 and will discuss again in Chapter 11, amount to aggregates for values that are functions.

7.1.4 Classification of Types

The terminology for types varies some from one language to another. This subsection presents definitions for the most common terms. Most languages provide built-in types similar to those supported in hardware by most processors: integers, characters, Booleans, and real (floating-point) numbers.

Booleans (sometimes called *logicals*) are typically implemented as single-byte quantities, with `1` representing `true` and `0` representing `false`. In a few languages and implementations, Booleans may be packed into arrays using only one bit per value. As noted in Section 6.1.2 ("Orthogonality"), C was historically unusual in omitting a Boolean type: where most languages would expect a Boolean value, C expected an integer, using zero for `false` and anything else for `true`. C99 introduced a new `_Bool` type, but it is effectively an integer that the compiler is permitted to store in a single bit. As noted in Section C-6.5.4, Icon replaces Booleans with a more general notion of *success* and *failure*.

Characters have traditionally been implemented as one-byte quantities as well, typically (but not always) using the ASCII encoding. More recent languages (e.g., Java and C#) use a two-byte representation designed to accommodate (the commonly used portion of) the *Unicode* character set. Unicode is an international standard designed to capture the characters of a wide variety of languages (see Sidebar 7.3). The first 128 characters of Unicode (`\u0000` through `\u007f`) are identical to ASCII. C and C++ provide both regular and "wide" characters, though for wide characters both the encoding and the actual width are implementation dependent. Fortran 2003 supports four-byte Unicode characters.

Numeric Types

A few languages (e.g., C and Fortran) distinguish between different lengths of integers and real numbers; most do not, and leave the choice of precision to the implementation. Unfortunately, differences in precision across language implementations lead to a lack of portability: programs that run correctly on one system may produce run-time errors or erroneous results on another. Java and C# are unusual in providing several lengths of numeric types, with a specified precision for each.

A few languages, including C, C++, C#, and Modula-2, provide both signed and unsigned integers (Modula-2 calls unsigned integers *cardinals*). A few languages (e.g., Fortran, C, Common Lisp, and Scheme) provide a built-in complex type, usually implemented as a pair of floating-point numbers that represent the real and imaginary Cartesian coordinates; other languages support these as a standard library class. A few languages (e.g., Scheme and Common Lisp) provide a built-in rational type, usually implemented as a pair of integers that represent the numerator and denominator. Most varieties of Lisp also support integers of arbitrary precision, as do most scripting languages; the implementation uses multiple words of memory where appropriate. Ada supports *fixed-point* types, which are represented internally by integers, but have an implied decimal point at a programmer-specified position among the digits. Several languages support

decimal types that use a base-10 encoding to avoid round-off anomalies in financial and human-centered arithmetic (see Sidebar 7.4).

Integers, Booleans, and characters are all examples of *discrete* types (also called *ordinal* types): the domains to which they correspond are countable (they have a one-to-one correspondence with some subset of the integers), and have a well-defined notion of predecessor and successor for each element other than the first and the last. (In most implementations the number of possible integers is finite, but this is usually not reflected in the type system.) Two varieties of user-defined types, enumerations and subranges, are also discrete. Discrete, rational, real, and

DESIGN & IMPLEMENTATION

7.3 Multilingual character sets

The *ISO 10646* international standard defines a *Universal Character Set (UCS)* intended to include all characters of all known human languages. (It also sets aside a "private use area" for such artificial [constructed] languages as Klingon, Tengwar, and Cirth [Tolkien Elvish]. Allocation of this private space is coordinated by a volunteer organization known as the ConScript Unicode Registry.) All natural languages currently employ codes in the 16-bit *Basic Multilingual Plane (BMP)*: `0x0000` through `0xfffd`.

Unicode is an expanded version of ISO 10646, maintained by an international consortium of software manufacturers. In addition to mapping tables, it covers such topics as rendering algorithms, directionality of text, and sorting and comparison conventions.

While recent languages have moved toward 16- or 32-bit internal character representations, these cannot be used for external storage—text files—without causing severe problems with backward compatibility. To accommodate Unicode without breaking existing tools, Ken Thompson in 1992 proposed a multibyte "expanding" code known as *UTF-8* (UCS/Unicode Transformation Format, 8-bit), and codified as a formal annex (appendix) to ISO 10646. UTF-8 characters occupy a maximum of 6 bytes—3 if they lie in the BMP, and only 1 if they are ordinary ASCII. The trick is to observe that ASCII is a 7-bit code; in any legacy text file the most significant bit of every byte is 0. In UTF-8 a most significant bit of 1 indicates a multibyte character. Two-byte codes begin with the bits `110`. Three-byte codes begin with `1110`. Second and subsequent bytes of multibyte characters always begin with `10`.

On some systems one also finds files encoded in one of ten variants of the older 8-bit ISO 8859 standard, but these are inconsistently rendered across platforms. On the web, non-ASCII characters are typically encoded with *numeric character references*, which bracket a Unicode value, written in decimal or hex, with an ampersand and a semicolon. The copyright symbol (©), for example, is `&#169;`. Many characters also have symbolic *entity names* (e.g., `&copy;`), but not all browsers support these.

complex types together constitute the *scalar* types. Scalar types are also sometimes called *simple* types.

Enumeration Types

Enumerations were introduced by Wirth in the design of Pascal. They facilitate the creation of readable programs, and allow the compiler to catch certain kinds of programming errors. An enumeration type consists of a set of named elements. In Pascal, one could write

EXAMPLE 7.9
Enumerations in Pascal

```
type weekday = (sun, mon, tue, wed, thu, fri, sat);
```

The values of an enumeration type are ordered, so comparisons are generally valid (`mon < tue`), and there is usually a mechanism to determine the predecessor or successor of an enumeration value (in Pascal, `tomorrow := succ(today)`). The

DESIGN & IMPLEMENTATION

7.4 Decimal types

A few languages, notably Cobol and PL/I, provide a *decimal* type for fixed-point representation of integer quantities. These types were designed primarily to exploit the *binary-coded decimal (BCD)* integer format supported by many traditional CISC machines. BCD devotes one *nibble* (four bits—half a byte) to each decimal digit. Machines that support BCD in hardware can perform arithmetic directly on the BCD representation of a number, without converting it to and from binary form. This capability is particularly useful in business and financial applications, which treat their data as both numbers and character strings.

With the growth in on-line commerce, the past few years have seen renewed interest in decimal arithmetic. The 2008 revision of the IEEE 754 floating-point standard includes decimal floating-point types in 32-, 64-, and 128-bit lengths. These represent both the mantissa (significant bits) and exponent in binary, but interpret the exponent as a power of ten, not a power of two. At a given length, values of decimal type have greater precision but smaller range than binary floating-point values. They are ideal for financial calculations, because they capture decimal fractions precisely. Designers hope the new standard will displace existing incompatible decimal formats, not only in hardware but also in software libraries, thereby providing the same portability and predictability that the original 754 standard provided for binary floating-point.

C# includes a 128-bit `decimal` type that is compatible with the new standard. Specifically, a C# decimal variable includes 96 bits of precision, a sign, and a decimal *scaling factor* that can vary between 10^{-28} and 10^{28}. IBM, for which business and financial applications have always been an important market, has included a hardware implementation of the standard (64- and 128-bit widths) in its pSeries RISC machines, beginning with the POWER6.

ordered nature of enumerations facilitates the writing of enumeration-controlled loops:

```
for today := mon to fri do begin ...
```

It also allows enumerations to be used to index arrays:

```
var daily_attendance : array [weekday] of integer;
```

EXAMPLE 7.10
Enumerations as constants

An alternative to enumerations, of course, is simply to declare a collection of constants:

```
const sun = 0; mon = 1; tue = 2; wed = 3; thu = 4; fri = 5; sat = 6;
```

In C, the difference between the two approaches is purely syntactic:

```
enum weekday {sun, mon, tue, wed, thu, fri, sat};
```

is essentially equivalent to

```
typedef int weekday;
const weekday sun = 0, mon = 1, tue = 2,
              wed = 3, thu = 4, fri = 5, sat = 6;
```

In Pascal and most of its descendants, however, the difference between an enumeration and a set of integer constants is much more significant: the enumeration is a full-fledged type, incompatible with integers. Using an integer or an enumeration value in a context expecting the other will result in a type clash error at compile time.

EXAMPLE 7.11
Converting to and from enumeration type

Values of an enumeration type are typically represented by small integers, usually a consecutive range of small integers starting at zero. In many languages these *ordinal values* are semantically significant, because built-in functions can be used to convert an enumeration value to its ordinal value, and sometimes vice versa. In Ada, these conversions employ the *attributes* `pos` and `val`: `weekday'pos(mon) = 1` and `weekday'val(1) = mon`.

EXAMPLE 7.12
Distinguished values for enums

Several languages allow the programmer to specify the ordinal values of enumeration types, if the default assignment is undesirable. In C, C++, and C#, one could write

```
enum arm_special_regs {fp = 7, sp = 13, lr = 14, pc = 15};
```

(The intuition behind these values is explained in Sections C-5.4.5 and C-9.2.2.)

In Ada this declaration would be written

```
type arm_special_regs is (fp, sp, lr, pc);      -- must be sorted
for arm_special_regs use (fp => 7, sp => 13, lr => 14, pc => 15);
```

EXAMPLE 7.13
Emulating distinguished enum values in Java

In recent versions of Java one can obtain a similar effect by giving values an extra field (here named `register`):

```
enum arm_special_regs { fp(7), sp(13), lr(14), pc(15);
    private final int register;
    arm_special_regs(int r) { register = r; }
    public int reg() { return register; }
}
...
int n = arm_special_regs.fp.reg();
```

As noted in Section 3.5.2, Pascal and C do not allow the same element name to be used in more than one enumeration type in the same scope. Java and C# do, but the programmer must identify elements using fully qualified names: `arm_special_regs.fp`. Ada relaxes this requirement by saying that element names are overloaded; the type prefix can be omitted whenever the compiler can infer it from context (Example 3.22). C++ historically mirrored C in prohibiting duplicate `enum` names. C++11 introduced a new variety of `enum` that mirrors Java and C# (Example 3.23).

Subrange Types

Like enumerations, subranges were first introduced in Pascal, and are found in many subsequent languages. A subrange is a type whose values compose a contiguous subset of the values of some discrete *base* type (also called the *parent* type). In Pascal and most of its descendants, one can declare subranges of integers, characters, enumerations, and even other subranges. In Pascal, subranges looked like this:

EXAMPLE 7.14
Subranges in Pascal

```
type test_score = 0..100;
     workday = mon..fri;
```

DESIGN & IMPLEMENTATION

7.5 Multiple sizes of integers

The space savings possible with (small-valued) subrange types in Pascal and Ada is achieved in several other languages by providing more than one size of built-in integer type. C and C++, for example, support integer arithmetic on signed and unsigned variants of `char`, `short`, `int`, `long`, and `long long` types, with monotonically nondecreasing sizes.[2]

2 More specifically, C requires ranges for these types corresponding to lengths of at least 1, 2, 2, 4, and 8 bytes, respectively. In practice, one finds implementations in which plain `int`s are 2, 4, or 8 bytes long, including some in which they are the same size as `short`s but shorter than `long`s, and some in which they are the same size as `long`s, and longer than `short`s.

EXAMPLE 7.15
Subranges in Ada

In Ada one would write

```
type test_score is new integer range 0..100;
subtype workday is weekday range mon..fri;
```

The `range...` portion of the definition in Ada is called a type *constraint*. In this example `test_score` is a *derived* type, incompatible with integers. The `workday` type, on the other hand, is a *constrained subtype*; `workdays` and `weekdays` can be more or less freely intermixed. The distinction between derived types and subtypes is a valuable feature of Ada; we will discuss it further in Section 7.2.1. ■

One could of course use integers to represent test scores, or a `weekday` to represent a `workday`. Using an explicit subrange has several advantages. For one thing, it helps to document the program. A comment could also serve as documentation, but comments have a bad habit of growing out of date as programs change, or of being omitted in the first place. Because the compiler analyzes a subrange declaration, it knows the expected range of subrange values, and can generate code to perform dynamic semantic checks to ensure that no subrange variable is ever assigned an invalid value. These checks can be valuable debugging tools. In addition, since the compiler knows the number of values in the subrange, it can sometimes use fewer bits to represent subrange values than it would need to use to represent arbitrary integers. In the example above, `test_score` values can be stored in a single byte.

EXAMPLE 7.16
Space requirements of subrange type

Most implementations employ the same bit patterns for integers and subranges, so subranges whose values are large require large storage locations, even if the number of distinct values is small. The following type, for example,

```
type water_temperature = 273..373;  (* degrees Kelvin *)
```

would be stored in at least two bytes. While there are only 101 distinct values in the type, the largest (373) is too large to fit in a single byte in its natural encoding. (An unsigned byte can hold values in the range $0..255$; a signed byte can hold values in the range $-128..127$.) ■

Composite Types

Nonscalar types are usually called *composite* types. They are generally created by applying a *type constructor* to one or more simpler types. *Options*, which we introduced in Example 7.6, are arguably the simplest composite types, serving only to add an extra "none of the above" to the values of some arbitrary base type. Other common composite types include records (structures), variant records (unions), arrays, sets, pointers, lists, and files. All but pointers and lists are easily described in terms of mathematical set operations (pointers and lists can be described mathematically as well, but the description is less intuitive).

Records (structs) were introduced by Cobol, and have been supported by most languages since the 1960s. A record consists of collection of *fields*, each of

which belongs to a (potentially different) simpler type. Records are akin to mathematical *tuples*; a record type corresponds to the Cartesian product of the types of the fields.

Variant records (unions) differ from "normal" records in that only one of a variant record's fields (or collections of fields) is valid at any given time. A variant record type is the *disjoint union* of its field types, rather than their Cartesian product.

Arrays are the most commonly used composite types. An array can be thought of as a function that maps members of an *index* type to members of a *component* type. Arrays of characters are often referred to as *strings*, and are often supported by special-purpose operations not available for other arrays.

Sets, like enumerations and subranges, were introduced by Pascal. A set type is the mathematical powerset of its base type, which must often be discrete. A variable of a set type contains a collection of distinct elements of the base type.

Pointers are l-values. A pointer value is a *reference* to an object of the pointer's base type. Pointers are often but not always implemented as addresses. They are most often used to implement *recursive* data types. A type T is recursive if an object of type T may contain one or more references to other objects of type T.

Lists, like arrays, contain a sequence of elements, but there is no notion of mapping or indexing. Rather, a list is defined recursively as either an empty list or a pair consisting of a head element and a reference to a sublist. While the length of an array must be specified at elaboration time in most (though not all) languages, lists are always of variable length. To find a given element of a list, a program must examine all previous elements, recursively or iteratively, starting at the head. Because of their recursive definition, lists are fundamental to programming in most functional languages.

Files are intended to represent data on mass-storage devices, outside the memory in which other program objects reside. Like arrays, most files can be conceptualized as a function that maps members of an index type (generally integer) to members of a component type. Unlike arrays, files usually have a notion of *current position*, which allows the index to be implied implicitly in consecutive operations. Files often display idiosyncrasies inherited from physical input/output devices. In particular, the elements of some files must be accessed in sequential order.

We will examine composite types in more detail in Chapter 8.

✓ CHECK YOUR UNDERSTANDING

1. What purpose(s) do types serve in a programming language?
2. What does it mean for a language to be *strongly typed*? *Statically typed*? What prevents, say, C from being strongly typed?

3. Name two programming languages that are strongly but dynamically typed.
4. What is a *type clash*?
5. Discuss the differences among the *denotational*, *structural*, and *abstraction-based* views of types.
6. What does it mean for a set of language features (e.g., a type system) to be *orthogonal*?
7. What are *aggregates*?
8. What are *option* types? What purpose do they serve?
9. What is *polymorphism*? What distinguishes its *parametric* and *subtype* varieties? What are *generics*?
10. What is the difference between *discrete* and *scalar* types?
11. Give two examples of languages that lack a Boolean type. What do they use instead?
12. In what ways may an enumeration type be preferable to a collection of named constants? In what ways may a subrange type be preferable to its base type? In what ways may a string be preferable to an array of characters?

7.2 Type Checking

In most statically typed languages, every definition of an object (constant, variable, subroutine, etc.) must specify the object's type. Moreover, many of the contexts in which an object might appear are also typed, in the sense that the rules of the language constrain the types that an object in that context may validly possess. In the subsections below we will consider the topics of *type equivalence*, *type compatibility*, and *type inference*. Of the three, type compatibility is the one of most concern to programmers. It determines when an object of a certain type can be used in a certain context. At a minimum, the object can be used if its type and the type expected by the context are equivalent (i.e., the same). In many languages, however, compatibility is a looser relationship than equivalence: objects and contexts are often compatible even when their types are different. Our discussion of type compatibility will touch on the subjects of type *conversion* (also called *casting*), which changes a value of one type into a value of another; type *coercion*, which performs a conversion automatically in certain contexts; and *nonconverting* type casts, which are sometimes used in systems programming to interpret the bits of a value of one type as if they represented a value of some other type.

Whenever an expression is constructed from simpler subexpressions, the question arises: given the types of the subexpressions (and possibly the type expected

by the surrounding context), what is the type of the expression as a whole? This question is answered by type inference. Type inference is often trivial: the sum of two integers is still an integer, for example. In other cases (e.g., when dealing with sets) it is a good bit trickier. Type inference plays a particularly important role in ML, Miranda, and Haskell, in which almost all type annotations are optional, and will be inferred by the compiler when omitted.

7.2.1 Type Equivalence

In a language in which the user can define new types, there are two principal ways of defining type equivalence. *Structural equivalence* is based on the *content* of type definitions: roughly speaking, two types are the same if they consist of the same components, put together in the same way. *Name equivalence* is based on the *lexical occurrence* of type definitions: roughly speaking, each definition introduces a new type. Structural equivalence is used in Algol-68, Modula-3, and (with various wrinkles) C and ML. Name equivalence appears in Java, C#, standard Pascal, and most Pascal descendants, including Ada.

The exact definition of structural equivalence varies from one language to another. It requires that one decide which potential differences between types are important, and which may be considered unimportant. Most people would probably agree that the format of a declaration should not matter—identical declarations that differ only in spacing or line breaks should still be considered equivalent. Likewise, in a Pascal-like language with structural equivalence,

EXAMPLE 7.17
Trivial differences in type

```
type R1 = record
    a, b : integer
end;
```

should probably be considered the same as

```
type R2 = record
    a : integer;
    b : integer
end;
```

But what about

```
type R3 = record
    b : integer;
    a : integer
end;
```

Should the reversal of the order of the fields change the type? ML says no; most languages say yes. ■

EXAMPLE 7.18
Other minor differences in type

In a similar vein, consider the following arrays, again in a Pascal-like notation:

```
type str = array [1..10] of char;

type str = array [0..9] of char;
```

Here the length of the array is the same in both cases, but the index values are different. Should these be considered equivalent? Most languages say no, but some (including Fortran and Ada) consider them compatible.

To determine if two types are structurally equivalent, a compiler can expand their definitions by replacing any embedded type names with their respective definitions, recursively, until nothing is left but a long string of type constructors, field names, and built-in types. If these expanded strings are the same, then the types are equivalent, and conversely. Recursive and pointer-based types complicate matters, since their expansion does not terminate, but the problem is not insurmountable; we consider a solution in Exercise 8.15.

EXAMPLE 7.19
The problem with structural equivalence

Structural equivalence is a straightforward but somewhat low-level, implementation-oriented way to think about types. Its principal problem is an inability to distinguish between types that the programmer may think of as distinct, but which happen by coincidence to have the same internal structure:

```
1.  type student = record
2.          name, address : string
3.          age : integer

4.  type school = record
5.          name, address : string
6.          age : integer

7.  x : student;
8.  y : school;
9.  . . .
10. x := y;                 -- is this an error?
```

Most programmers would probably want to be informed if they accidentally assigned a value of type `school` into a variable of type `student`, but a compiler whose type checking is based on structural equivalence will blithely accept such an assignment.

Name equivalence is based on the assumption that if the programmer goes to the effort of writing two type definitions, then those definitions are probably meant to represent different types. In the example above, variables `x` and `y` will be considered to have different types under name equivalence: `x` uses the type declared at line 1; `y` uses the type declared at line 4.

Variants of Name Equivalence

EXAMPLE 7.20
Alias types

One subtlety in the use of name equivalence arises in the simplest of type declarations:

```
type new_type = old_type;           (* Algol family syntax *)

typedef old_type new_type;          /* C family syntax */
```

Here `new_type` is said to be an *alias* for `old_type`. Should we treat them as two names for the same type, or as names for two different types that happen to have the same internal structure? The "right" approach may vary from one program to another. ■

EXAMPLE 7.21
Semantically equivalent alias types

Users of any Unix-like system will be familiar with the notion of *permission* bits on files. These specify whether the file is readable, writable, and/or executable by its owner, group members, or others. Within the system libraries, the set of permissions for a file is represented as a value of type `mode_t`. In C, this type is commonly defined as an alias for the predefined 16-bit unsigned integer type:

```
typedef uint16_t mode_t;
```

While C uses structural equivalence for scalar types,[3] we can imagine the issue that would arise if it used name equivalence uniformly. By convention, permission sets are manipulated using bitwise integer operators:

```
mode_t my_permissions = S_IRUSR | S_IWUSR | S_IRGRP;
/* I can read and write; members of my group can read. */
...
if (my_permissions & S_IWUSR) ...
```

This convention depends on the equivalence of `mode_t` and `uint16_t`. One could ask programmers to convert `mode_t` objects explicitly to `uint_16` before applying an integer operator—or even suggest that `mode_t` be an abstract type, with `insert`, `remove`, and `lookup` operations that hide the internal representation—but C programmers would probably regard either of these options as unnecessarily cumbersome: in "systems" code, it seems reasonable to treat `mode_t` and `uint16_t` the same. ■

EXAMPLE 7.22
Semantically distinct alias types

Unfortunately, there are other times when aliased types should probably not be the same:

```
type celsius_temp = real;
     fahrenheit_temp = real;
var  c : celsius_temp;
     f : fahrenheit_temp;
...
f := c;             (* this should probably be an error *)
```

■

A language in which aliased types are considered distinct is said to have *strict name equivalence*. A language in which aliased types are considered equivalent is said to have *loose name equivalence*. Most Pascal-family languages use loose name equivalence. Ada achieves the best of both worlds by allowing the programmer to indicate whether an alias represents a *derived* type or a *subtype*. A subtype is

EXAMPLE 7.23
Derived types and subtypes in Ada

---

3 Ironically, it uses name equivalence for `structs`.

compatible with its base (parent) type; a derived type is incompatible. (Subtypes of the same base type are also compatible with each other.) Our examples above would be written

```
subtype mode_t is integer range 0..2**16-1;    -- unsigned 16-bit integer
...
type celsius_temp is new integer;
type fahrenheit_temp is new integer;
```

One way to think about the difference between strict and loose name equivalence is to remember the distinction between declarations and definitions (Section 3.3.3). Under strict name equivalence, a declaration `type A = B` is considered a definition. Under loose name equivalence it is merely a declaration; `A` shares the definition of `B`.

EXAMPLE 7.24

Name vs structural equivalence

Consider the following example:

```
1. type cell    = . . .              -- whatever
2. type alink   = pointer to cell
3. type blink   = alink
4. p, q : pointer to cell
5. r    : alink
6. s    : blink
7. t    : pointer to cell
8. u    : alink
```

Here the declaration at line 3 is an alias; it defines blink to be "the same as" alink. Under strict name equivalence, line 3 is both a declaration and a definition, and blink is a new type, distinct from alink. Under loose name equivalence, line 3 is just a declaration; it uses the definition at line 2.

Under strict name equivalence, p and q have the same type, because they both use the *anonymous* (unnamed) type definition on the right-hand side of line 4, and r and u have the same type, because they both use the definition at line 2. Under loose name equivalence, r, s, and u all have the same type, as do p and q. Under structural equivalence, all six of the variables shown have the same type, namely pointer to whatever cell is.

Both structural and name equivalence can be tricky to implement in the presence of separate compilation. We will return to this issue in Section 15.6.

Type Conversion and Casts

EXAMPLE 7.25

Contexts that expect a given type

In a language with static typing, there are many contexts in which values of a specific type are expected. In the statement

```
a := expression
```

we expect the right-hand side to have the same type as a. In the expression

```
a + b
```

the overloaded + symbol designates either integer or floating-point addition; we therefore expect either that a and b will both be integers, or that they will both be reals. In a call to a subroutine,

```
foo(arg1, arg2, . . . , argN)
```

we expect the types of the arguments to match those of the formal parameters, as declared in the subroutine's header.

Suppose for the moment that we require in each of these cases that the types (expected and provided) be exactly the same. Then if the programmer wishes to use a value of one type in a context that expects another, he or she will need to specify an explicit *type conversion* (also sometimes called a type *cast*). Depending on the types involved, the conversion may or may not require code to be executed at run time. There are three principal cases:

1. The types would be considered structurally equivalent, but the language uses name equivalence. In this case the types employ the same low-level representation, and have the same set of values. The conversion is therefore a purely conceptual operation; no code will need to be executed at run time.
2. The types have different sets of values, but the intersecting values are represented in the same way. One type may be a subrange of the other, for example, or one may consist of two's complement signed integers, while the other is unsigned. If the provided type has some values that the expected type does not, then code must be executed at run time to ensure that the current value is among those that are valid in the expected type. If the check fails, then a dynamic semantic error results. If the check succeeds, then the underlying representation of the value can be used, unchanged. Some language implementations may allow the check to be disabled, resulting in faster but potentially unsafe code.
3. The types have different low-level representations, but we can nonetheless define some sort of correspondence among their values. A 32-bit integer, for example, can be converted to a double-precision IEEE floating-point number with no loss of precision. Most processors provide a machine instruction to effect this conversion. A floating-point number can be converted to an integer by rounding or truncating, but fractional digits will be lost, and the conversion will overflow for many exponent values. Again, most processors provide a machine instruction to effect this conversion. Conversions between different lengths of integers can be effected by discarding or sign-extending high-order bytes.

EXAMPLE 7.26
Type conversions in Ada

We can illustrate these options with the following examples of type conversions in Ada:

```
n : integer;          -- assume 32 bits
r : long_float;       -- assume IEEE double-precision
t : test_score;       -- as in Example 7.15
c : celsius_temp;     -- as in Example 7.23
```

```
...
t := test_score(n);    -- run-time semantic check required
n := integer(t);       -- no check req.; every test_score is an int
r := long_float(n);    -- requires run-time conversion
n := integer(r);       -- requires run-time conversion and check
n := integer(c);       -- no run-time code required
c := celsius_temp(n);  -- no run-time code required
```

In each of the six assignments, the name of a type is used as a pseudofunction that performs a type conversion. The first conversion requires a run-time check to ensure that the value of `n` is within the bounds of a `test_score`. The second conversion requires no code, since every possible value of `t` is acceptable for `n`. The third and fourth conversions require code to change the low-level representation of values. The fourth conversion also requires a semantic check. It is generally understood that converting from a floating-point value to an integer results in the loss of fractional digits; this loss is not an error. If the conversion results in integer overflow, however, an error needs to result. The final two conversions require no run-time code; the `integer` and `celsius_temp` types (at least as we have defined them) have the same sets of values and the same underlying representation. A purist might say that `celsius_temp` should be defined as `new integer range -273..integer'last`, in which case a run-time semantic check would be required on the final conversion. ■

EXAMPLE 7.27
Type conversions in C

A type conversion in C (what C calls a type cast) is specified by using the name of the desired type, in parentheses, as a prefix operator:

```
r = (float) n;  /* generates code for run-time conversion */
n = (int) r;    /* also run-time conversion, with no overflow check */
```

C and its descendants do not by default perform run-time checks for arithmetic overflow on any operation, though such checks can be enabled if desired in C#. ■

Nonconverting Type Casts Occasionally, particularly in systems programs, one needs to change the type of a value *without* changing the underlying implementation; in other words, to interpret the bits of a value of one type as if they were another type. One common example occurs in memory allocation algorithms, which use a large array of bytes to represent a heap, and then reinterpret portions of that array as pointers and integers (for bookkeeping purposes), or as various user-allocated data structures. Another common example occurs in high-performance numeric software, which may need to reinterpret a floating-point number as an integer or a record, in order to extract the exponent, significand, and sign fields. These fields can be used to implement special-purpose algorithms for square root, trigonometric functions, and so on.

A change of type that does not alter the underlying bits is called a *nonconverting type cast*, or sometimes a *type pun*. It should not be confused with

EXAMPLE 7.28
Unchecked conversions in Ada

use of the term *cast* for conversions in languages like C. In Ada, nonconverting casts can be effected using instances of a built-in generic subroutine called `unchecked_conversion`:

```
-- assume 'float' has been declared to match IEEE single-precision
function cast_float_to_int is
    new unchecked_conversion(float, integer);
function cast_int_to_float is
    new unchecked_conversion(integer, float);
...
f := cast_int_to_float(n);
n := cast_float_to_int(f);
```

EXAMPLE 7.29
Conversions and nonconverting casts in C++

C++ inherits the casting mechanism of C, but also provides a family of semantically cleaner alternatives. Specifically, `static_cast` performs a type conversion, `reinterpret_cast` performs a nonconverting type cast, and `dynamic_cast` allows programs that manipulate pointers of polymorphic types to perform assignments whose validity cannot be guaranteed statically, but can be checked at run time (more on this in Chapter 10). Syntax for each of these is that of a generic function:

DESIGN & IMPLEMENTATION

7.6 Nonconverting casts

C programmers sometimes attempt a nonconverting type cast (type pun) by taking the address of an object, converting the type of the resulting pointer, and then dereferencing:

```
r = *((float *) &n);
```

This arcane bit of hackery usually incurs no run-time cost, because most (but not all!) implementations use the same representation for pointers to integers and pointers to floating-point values—namely, an address. The ampersand operator (`&`) means "address of," or "pointer to." The parenthesized `(float *)` is the type name for "pointer to float" (float is a built-in floating-point type). The prefix `*` operator is a pointer dereference. The overall construct causes the compiler to interpret the bits of `n` as if it were a `float`. The reinterpretation will succeed only if `n` is an l-value (has an address), and `int`s and `float`s have the same size (again, this second condition is often but not always true in C). If `n` does not have an address then the compiler will announce a static semantic error. If `int` and `float` do not occupy the same number of bytes, then the effect of the cast may depend on a variety of factors, including the relative size of the objects, the alignment and "endian-ness" of memory (Section C-5.2), and the choices the compiler has made regarding what to place in adjacent locations in memory. Safer and more portable nonconverting casts can be achieved in C by means of `union`s (variant records); we consider this option in Exercise C-8.24.

```
double d = ...
int n = static_cast<int>(d);
```

There is also a `const_cast` that can be used to remove read-only qualification. C-style type casts in C++ are defined in terms of `const_cast`, `static_cast`, and `reinterpret_cast`; the precise behavior depends on the source and target types.

Any nonconverting type cast constitutes a dangerous subversion of the language's type system. In a language with a weak type system such subversions can be difficult to find. In a language with a strong type system, the use of explicit nonconverting type casts at least labels the dangerous points in the code, facilitating debugging if problems arise.

7.2.2 Type Compatibility

Most languages do not require equivalence of types in every context. Instead, they merely say that a value's type must be *compatible* with that of the context in which it appears. In an assignment statement, the type of the right-hand side must be compatible with that of the left-hand side. The types of the operands of + must both be compatible with some common type that supports addition (integers, real numbers, or perhaps strings or sets). In a subroutine call, the types of any arguments passed into the subroutine must be compatible with the types of the corresponding formal parameters, and the types of any formal parameters passed back to the caller must be compatible with the types of the corresponding arguments.

The definition of type compatibility varies greatly from language to language. Ada takes a relatively restrictive approach: an Ada type `S` is compatible with an expected type `T` if and only if (1) `S` and `T` are equivalent, (2) one is a subtype of the other (or both are subtypes of the same base type), or (3) both are arrays, with the same numbers and types of elements in each dimension. Pascal was only slightly more lenient: in addition to allowing the intermixing of base and subrange types, it allowed an integer to be used in a context where a real was expected.

Coercion

Whenever a language allows a value of one type to be used in a context that expects another, the language implementation must perform an automatic, implicit conversion to the expected type. This conversion is called a *type coercion*. Like the explicit conversions of Section 7.2.1, coercion may require run-time code to perform a dynamic semantic check or to convert between low-level representations.

EXAMPLE 7.30
Coercion in C

C, which has a relatively weak type system, performs quite a bit of coercion. It allows values of most numeric types to be intermixed in expressions, and will coerce types back and forth "as necessary." Consider the following declarations:

```
short int s;
unsigned long int l;
char c;     /* may be signed or unsigned -- implementation-dependent */
float f;    /* usually IEEE single-precision */
double d;   /* usually IEEE double-precision */
```

Suppose that these variables are 16, 32, 8, 32, and 64 bits in length, respectively—as is common on 32-bit machines. Coercion may have a variety of effects when a variable of one type is assigned into another:

```
s = l;  /* l's low-order bits are interpreted as a signed number. */
l = s;  /* s is sign-extended to the longer length, then
           its bits are interpreted as an unsigned number. */
s = c;  /* c is either sign-extended or zero-extended to s's length;
           the result is then interpreted as a signed number. */
f = l;  /* l is converted to floating-point.  Since f has fewer
           significant bits, some precision may be lost. */
d = f;  /* f is converted to the longer format; no precision is lost. */
f = d;  /* d is converted to the shorter format; precision may be lost.
           If d's value cannot be represented in single-precision, the
           result is undefined, but NOT a dynamic semantic error. */
```

Coercion is a somewhat controversial subject in language design. Because it allows types to be mixed without an explicit indication of intent on the part of the programmer, it represents a significant weakening of type security. At the same time, some designers have argued that coercions are a natural way in which to support abstraction and program extensibility, by making it easier to use new types in conjunction with existing ones. This extensibility argument is particularly compelling in scripting languages (Chapter 14), which are dynamically typed and emphasize ease of programming. Most scripting languages support a wide variety of coercions, though there is some variation: Perl will coerce almost anything; Ruby is much more conservative.

Among statically typed languages, there is even more variety. Ada coerces nothing but explicit constants, subranges, and in certain cases arrays with the same type of elements. Pascal would coerce integers to floating-point in expressions and assignments. Fortran will also coerce floating-point values to integers in assignments, at a potential loss of precision. C will perform these same coercions on arguments to functions.

Some compiled languages even support coercion on arrays and records. Fortran 90 permits this whenever the expected and actual types have the same *shape*. Two arrays have the same shape if they have the same number of dimensions, each dimension has the same size (i.e., the same number of elements), and the individual elements have the same shape. Two records have the same shape if they have the same number of fields, and corresponding fields, in order, have the same shape. Field *names* do not matter, nor do the actual high and low bounds of array dimensions. Ada's coercion rules for arrays are roughly equivalent to those of Fortran 90. C provides no operations that take an entire array as an operand.

C does, however, allow arrays and *pointers* to be intermixed in many cases; we will discuss this unusual form of type compatibility further in Section 8.5.1. Neither Ada nor C allows records (structures) to be intermixed unless their types are name equivalent.

C++ provides what may be the most extreme example of coercion in a statically typed language. In addition to a rich set of built-in rules, C++ allows the programmer to define coercion operations to and from existing types when defining a new type (a *class*). The rules for applying these operations interact in complicated ways with the rules for resolving overloading (Section 3.5.2); they add significant flexibility to the language, but are one of the most difficult C++ features to understand and use correctly.

Overloading and Coercion

We have noted (in Section 3.5) that overloading and coercion (as well as various forms of polymorphism) can sometimes be used to similar effect. It is worth elaborating on the distinctions here. An overloaded name can refer to more than one object; the ambiguity must be resolved by context. Consider the addition of numeric quantities. In the expression `a + b`, `+` may refer to either the integer or the floating-point addition operation. In a language without coercion, `a` and `b` must either both be integer or both be real; the compiler chooses the appropriate interpretation of `+` depending on their type. In a language with coercion, `+` refers to the floating-point addition operation if either `a` or `b` is real; otherwise it refers to the integer addition operation. If only one of `a` and `b` is real, the other is coerced to match. One could imagine a language in which `+` was not overloaded, but rather referred to floating-point addition in all cases. Coercion could still allow `+` to take integer arguments, but they would always be converted to real. The problem with this approach is that conversions from integer to floating-point format take a non-negligible amount of time, especially on machines without hardware conversion instructions, and floating-point addition is significantly more expensive than integer addition. ■

EXAMPLE 7.31
Coercion vs overloading of addends

In most languages, literal constants (e.g., numbers, character strings, the empty set `[ ]` or the null pointer `nil`) can be intermixed in expressions with values of many types. One might say that constants are overloaded: `nil` for example might be thought of as referring to the null pointer value for whatever type is needed in the surrounding context. More commonly, however, constants are simply treated as a special case in the language's type-checking rules. Internally, the compiler considers a constant to have one of a small number of built-in "constant types" (int const, real const, string, null), which it then coerces to some more appropriate type as necessary, even if coercions are not supported elsewhere in the language. Ada formalizes this notion of "constant type" for numeric quantities: an integer constant (one without a decimal point) is said to have type `universal_integer`; a real-number constant (one with an embedded decimal point and/or an exponent) is said to have type `universal_real`. The `universal_integer` type is compatible with any integer type; `universal_real` is compatible with any fixed-point or floating-point type.

Universal Reference Types

For systems programming, or to facilitate the writing of general-purpose *container* (*collection*) objects (lists, stacks, queues, sets, etc.) that hold references to other objects, several languages provide a *universal* reference type. In C and C++, this type is called `void*`. In Clu it is called `any`; in Modula-2, `address`; in Modula-3, `refany`; in Java, `Object`; in C#, `object`. Arbitrary l-values can be assigned into an object of universal reference type, with no concern about type safety: because the type of the object referred to by a universal reference is unknown, the compiler will not allow any operations to be performed on that object. Assignments back into objects of a particular reference type (e.g., a pointer to a programmer-specified record type) are a bit trickier, if type safety is to be maintained. We would not want a universal reference to a floating-point number, for example, to be assigned into a variable that is supposed to hold a reference to an integer, because subsequent operations on the "integer" would interpret the bits of the object incorrectly. In object-oriented languages, the question of how to ensure the validity of a universal-to-specific assignment generalizes to the question of how to ensure the validity of any assignment in which the type of the object on left-hand side supports operations that the object on the right-hand side may not.

One way to ensure the safety of universal to specific assignments (or, in general, less specific to more specific assignments) is to make objects self-descriptive—that is, to include in the representation of each object a *tag* that indicates its type. This approach is common in object-oriented languages, which generally need it for dynamic method binding. Type tags in objects can consume a nontrivial amount of space, but allow the implementation to prevent the assignment of an object of one type into a variable of another. In Java and C#, a universal to specific assignment requires a type cast, and will generate an exception if the universal reference does not refer to an object of the casted type. In Eiffel, the equivalent operation uses a special assignment operator (`?=` instead of `:=`); in C++ it uses a `dynamic_cast` operation.

EXAMPLE 7.32
Java container of `Object`

In early versions of Java and C#, programmers would often create container classes that held objects of the universal reference class (`Object` or `object`, respectively). This idiom has become less common with the introduction of generics (to be discussed in Section 7.3.1), but it is still occasionally used for containers that hold objects of more than one class. When an object is removed from such a container, it must be assigned (with a type cast) into a variable of an appropriate class before anything interesting can be done with it:

```
import java.util.*;     // library containing Stack container class
...
Stack myStack = new Stack();
String s = "Hi, Mom";
Foo f = new Foo();      // f is of user-defined class type Foo
...
```

```
myStack.push(s);
myStack.push(f);        // we can push any kind of object on a stack
...
s = (String) myStack.pop();
    // type cast is required, and will generate an exception at run
    // time if element at top-of-stack is not a string
```

In a language without type tags, the assignment of a universal reference into an object of a specific reference type cannot be checked, because objects are not self-descriptive: there is no way to identify their type at run time. The programmer must therefore resort to an (unchecked) type conversion.

7.2.3 Type Inference

We have seen how type checking ensures that the components of an expression (e.g., the arguments of a binary operator) have appropriate types. But what determines the type of the overall expression? In many cases, the answer is easy. The result of an arithmetic operator usually has the same type as the operands (possibly after coercing one of them, if their types were not the same). The result of a comparison is usually Boolean. The result of a function call has the type declared in the function's header. The result of an assignment (in languages in which assignments are expressions) has the same type as the left-hand side. In a few cases, however, the answer is not obvious. Operations on subranges and composite objects, for example, do not necessarily preserve the types of the operands. We examine these cases in the remainder of this subsection. In the following section, we consider a more elaborate form of type inference found in ML, Miranda, and Haskell.

Subranges and Sets

EXAMPLE 7.33
Inference of subrange types

For arithmetic operators, a simple example of inference arises when one or more operands have subrange types. Given the following Pascal definitions, for example,

```
type Atype = 0..20;
     Btype = 10..20;
var  a : Atype;
     b : Btype;
```

what is the type of `a + b`? Certainly it is neither `Atype` nor `Btype`, since the possible values range from 10 to 40. One could imagine it being a new anonymous subrange type with 10 and 40 as bounds. The usual answer is to say that the result of any arithmetic operation on a subrange has the subrange's base type—in this case, integer.

If the result of an arithmetic operation is assigned into a variable of a subrange type, then a dynamic semantic check may be required. To avoid the expense of some unnecessary checks, a compiler may keep track at compile time of the largest and smallest possible values of each expression, in essence computing the anonymous `10...40` type. More sophisticated techniques can be used to eliminate many checks in loops; we will consider these in Section C-17.5.2. ■

EXAMPLE 7.34
Type inference for sets

Operations with type implications also occur when manipulating sets. Pascal and Modula, for example, supported union (`+`), intersection (`*`), and difference (`-`) on sets of discrete values. Set operands were said to have compatible types if their elements had the same base type `T`. The result of a set operation was then of type `set of T`. As with subranges, a compiler could avoid the need for run-time bounds checks in certain cases by keeping track of the minimum and maximum possible members of the set expression. ■

Declarations

Ada was among the first languages to make the index of a `for` loop a new, local variable, accessible only in the loop. Rather than require the programmer to specify the type of this variable, the language implicitly assigned it the base type of the expressions provided as bounds for the loop.

Extensions of this idea appear in several more recent languages, including Scala, C# 3.0, C++11, Go, and Swift, all of which allow the programmer to omit type information from a variable declaration when the intent of the declaration can be inferred from context. In C#, for example, one can write

EXAMPLE 7.35
`var` declarations in C#

```
var i = 123;                                // equiv. to int i = 123;
var map = new Dictionary<string, int>();  // equiv. to
    // Dictionary<string, int> map = new Dictionary<string, int>();
```

Here the (easily determined) type of the right-hand side of the assignment can be used to infer the variable's type, freeing us from the need to declare it explicitly. We can achieve a similar effect in C++ with the `auto` keyword; in Scala we simply omit the type name when declaring an initialized variable or constant. ■

EXAMPLE 7.36
Avoiding messy declarations

The convenience of inference increases with complex declarations. Suppose, for example, that we want to perform what mathematicians call a *reduction* on the elements of a list—a "folding together" of values using some binary function. Using C++ lambda syntax (Section 3.6.4), we might write

```
auto reduce = [](list<int> L, int f(int, int), int s) {
        // the initial value of s should be the identity element for f
        for (auto e : L) {
            s = f(e, s);
        }
        return s;
    };
}
...
int sum = reduce(my_list, [](int a, int b){return a+b;}, 0);
int product = reduce(my_list, [](int a, int b){return a*b;}, 1);
```

Here the `auto` keyword allows us to omit what would have been a rather daunting indication of type:

```
int (*reduce) (list<int>, int (*)(int, int), int) = ...
    = [](list<int> L, int f(int, int), int s){...
```

EXAMPLE 7.37
`decltype` in C++11

C++ in fact goes one step further, with a `decltype` keyword that can be used to match the type of any existing expression. The `decltype` keyword is particularly handy in templates, where it is sometimes impossible to provide an appropriate static type name. Consider, for example, a generic arithmetic package, parameterized by operand types `A` and `B`:

```
template <typename A, typename B>
...
    A a;  B b;
    decltype(a + b) sum;
```

Here the type of `sum` depends on the types of `A` and `B` under the C++ coercion rules. If `A` and `B` are both `int`, for example, then `sum` will be an `int`. If one of `A` and `B` is `double` and the other is `int`, then `sum` will be a `double`. With appropriate (user-provided) coercion rules, `sum` might be inferred to have a complex (real + imaginary) or arbitrary-precision ("bignum") type.

7.2.4 Type Checking in ML

The most sophisticated form of type inference occurs in the ML family of functional languages, including Haskell, F#, and the OCaml and SML dialects of ML itself. Programmers have the option of declaring the types of objects in these languages, in which case the compiler behaves much like that of a more traditional statically typed language. As we noted near the beginning of Section 7.1, however, programmers may also choose not to declare certain types, in which case the compiler will infer them, based on the known types of literal constants, the explicitly declared types of any objects that have them, and the syntactic structure

of the program. ML-style type inference is the invention of the language's creator, Robin Milner.[4]

The key to the inference mechanism is to *unify* the (partial) type information available for two expressions whenever the rules of the type system say that their types must be the same. Information known about each is then known about the other as well. Any discovered inconsistencies are identified as static semantic errors. Any expression whose type remains incompletely specified after inference is automatically polymorphic; this is the *implicit parametric polymorphism* referred to in Section 7.1.2. ML family languages also incorporate a powerful run-time pattern-matching facility and several unconventional structured types, including ordered tuples, (unordered) records, lists, a `datatype` mechanism that subsumes unions and recursive types, and a rich module system with inheritance (type extension) and explicit parametric polymorphism (generics). We will consider ML types in more detail in Section 11.4.

EXAMPLE 7.38
Fibonacci function in OCaml

The following is an OCaml version of the tail-recursive Fibonacci function introduced in Example 6.87:

```
1.  let fib n =
2.    let rec fib_helper n1 n2 i =
3.      if i = n then n2
4.      else fib_helper n2 (n1 + n2) (i + 1) in
5.    fib_helper 0 1 0;;
```

The inner `let` construct introduces a nested scope: function `fib_helper` is nested inside `fib`. The body of the outer function, `fib`, is the expression `fib_helper 0 1 0`. The body of `fib_helper` is an `if... then ... else` expression; it evaluates to either `n2` or to `fib_helper n2 (n1 + n2) (i + 1)`, depending on whether the third argument to `fib_helper` is `n` or not. The keyword `rec` indicates that `fib_helper` is recursive, so its name should be made available within its own body—not just in the body of the `let`.

Given this function definition, an OCaml compiler will reason roughly as follows: Parameter `i` of `fib_helper` must have type `int`, because it is added to 1 at line 4. Similarly, parameter `n` of `fib` must have type `int`, because it is compared to `i` at line 3. In the call to `fib_helper` at line 5, the types of all three arguments are `int`, and since this is the only call, the types of `n1` and `n2` are `int`. Moreover the type of `i` is consistent with the earlier inference, namely `int`, and the types of the arguments to the recursive call at line 4 are similarly consistent. Since `fib_helper` returns `n2` at line 3, the result of the call at line 5 will be an `int`. Since `fib` immediately returns this result as its own result, the return type of `fib` is `int`. ■

4 Robin Milner (1934–2010), of Cambridge University's Computer Laboratory, was responsible not only for the development of ML and its type system, but for the Logic of Computable Functions, which provides a formal basis for machine-assisted proof construction, and the Calculus of Communicating Systems, which provides a general theory of concurrency. He received the ACM Turing Award in 1991.

EXAMPLE 7.39
Checking with explicit types

Of course, if any of our functions or parameters had been declared with explicit types, these would have been checked for consistency with all the other evidence. We might, for example, have begun with

```
let fib (n : int) : int = ...
```

to indicate that the function's parameter and return value were both expected to be integers. In a sense, explicit type declarations in OCaml serve as compiler-checked documentation. ■

EXAMPLE 7.40
Expression types

Because OCaml is a functional language, every construct is an expression. The compiler infers a type for every object and every expression. Because functions are first-class values, they too have types. The type of `fib` above is `int -> int`; that is, a function from integers to integers. The type of `fib_helper` is `int -> int -> int -> int`; that is, a function that takes three integer arguments and produces an integer result. Note that parentheses are generally omitted in both declarations of and calls to multiargument functions. If we had said

```
let rec fib_helper (n1, n2, i) =
  if i = n then n2
  else fib_helper (n2, n1+n2, i+1) in ...
```

then `fib_helper` would have accepted a *single* expression—a three-element *tuple*—as argument.[5] ■

EXAMPLE 7.41
Type inconsistency

Type correctness in the ML family amounts to what we might call type *consistency*: a program is type correct if the type checking algorithm can reason out a unique type for every expression, with no contradictions and no ambiguous occurrences of overloaded names. If the programmer uses an object inconsistently, the compiler will complain. In a program containing the following definition,

```
let circum r = r *. 2.0 *. 3.14159;;
```

the compiler will infer that `circum`'s parameter is of type `float`, because it is combined with the floating-point constants `2.0` and `3.14159`, using `*.`, the floating-point multiplication operator (here the dot is part of the operator name; there is a separate integer multiplication operator, `*`). If we attempt to apply `circum` to an integer argument, the compiler will produce a type clash error message. ■

Though the language is usually compiled in production environments, the standard OCaml distribution also includes an interactive interpreter. The programmer can interact with the interpreter "on line," giving it input a line at a

5 Multiple arguments are actually somewhat more complicated than suggested here, due to the fact that functions in OCaml are automatically *curried*; see Section 11.6 for more details.

time. The interpreter processes this input incrementally, generating an intermediate representation for each source code function, and producing any appropriate static error messages. This style of interaction blurs the traditional distinction between interpretation and compilation. While the language implementation remains active during program execution, it performs all possible semantic checks—everything that the production compiler would check—before evaluating a given program fragment.

EXAMPLE 7.42
Polymorphic functions

In comparison to languages in which programmers must declare all types explicitly, the type inference of ML-family languages has the advantage of brevity and convenience for interactive use. More important, it provides a powerful form of implicit parametric polymorphism more or less for free. While all uses of objects in an OCaml program must be consistent, they do not have to be completely specified. Consider the OCaml function shown in Figure 7.1. Here the equality test (`=`) is a built-in polymorphic function of type `'a -> 'a -> bool`; that is, a function that takes two arguments of the same type and produces a Boolean result. The token `'a` is called a *type variable*; it stands for any type,

DESIGN & IMPLEMENTATION

7.7 Type classes for overloaded functions in Haskell

In the OCaml code of Figure 7.1, parameters `x`, `p`, and `q` must support the equality operator (`=`). OCaml makes this easy by allowing anything to be compared for equality, and then checking at run time to make sure that the comparison actually makes sense. An attempt to compare two functions, for example, will result in a run-time error. This is unfortunate, given that most other type checking in OCaml (and in other ML-family languages) can happen at compile time. In a similar vein, OCaml provides a built-in definition of ordering (`<`, `>`, `<=`, and `>=`) on almost all types, even when it doesn't make sense, so that the programmer can create polymorphic functions like `min`, `max`, and `sort`, which require it. A function like `average`, which might plausibly work in a polymorphic fashion for all numeric types (presumably with roundoff for integers) cannot be defined in OCaml: each numeric type has its own addition and division operations; there is no operator overloading.

Haskell overcomes these limitations using the machinery of *type classes*. As mentioned in Example 3.28, these explicitly identify the types that support a particular overloaded function or set of functions. Elements of any type in the `Ord` class, for example, support the `<`, `>`, `<=`, and `>=` operations. Elements of any type in the `Enum` class are countable; `Num` types support addition, subtraction, and multiplication; `Fractional` and `Real` types additionally support division. In the Haskell equivalent of the code in Figure 7.1, parameters `x`, `p`, and `q` would be inferred to belong to some type in the class `Eq`. Elements of an array passed to `sort` would be inferred to belong to some type in the class `Ord`. Type consistency in Haskell can thus be verified entirely at compile time: there is no need for run-time checks.

```
let compare x p q =
  if x = p then
    if x = q then "both"
      else "first"
    else
      if x = q then "second"
      else "neither";;
```

Figure 7.1 An OCaml program to illustrate checking for type consistency.

and takes, implicitly, the role of an explicit type parameter in a generic construct (Sections 7.3.1 and 10.1.1). Every instance of `'a` in a given call to `=` must represent the same type, but instances of `'a` in different calls can be different. Starting with the type of `=`, an OCaml compiler can reason that the type of `compare` is `'a -> 'a -> 'a -> string`. Thus `compare` is polymorphic; it does not depend on the types of `x`, `p`, and `q`, so long as they are all the same. The key point to observe is that the programmer did not have to do anything special to make `compare` polymorphic: polymorphism is a natural consequence of ML-style type inference.

Type Checking

An OCaml compiler verifies type consistency with respect to a well-defined set of constraints. For example,

- All occurrences of the same identifier (subject to scope rules) have the same type.
- In an `if...then...else` expression, the condition is of type `bool`, and the `then` and `else` clauses have the same type.
- A programmer-defined function has type `'a -> 'b -> ... -> 'r`, where `'a`, `'b`, and so forth are the types of the function's parameters, and `'r` is the type of its result (the expression that forms its body).
- When a function is applied (called), the types of the arguments that are passed are the same as the types of the parameters in the function's definition. The type of the application (i.e., the expression constituted by the call) is the same as the type of the result in the function's definition.

In any case where two types *A* and *B* are required to be "the same," the OCaml compiler must *unify* what it knows about *A* and *B* to produce a (potentially more detailed) description of their common type. The inference can work in either direction, or both directions at once. For example, if the compiler has determined that `E1` is an expression of type `'a * int` (that is, a two-element tuple whose second element is known to be an integer), and that `E2` is an expression of type `string * 'b`, then in the expression `if x then E1 else E2`, it can infer that `'a` is `string` and `'b` is `int`. Thus `x` is of type `bool`, and `E1` and `E2` are of type `string * int`.

EXAMPLE 7.43

A simple instance of unification

DESIGN & IMPLEMENTATION

7.8 Unification

Unification is a powerful technique. In addition to its role in type inference (which also arises in the templates [generics] of C++), unification plays a central role in the computational model of Prolog and other logic languages. We will consider this latter role in Section 12.1. In the general case the cost of unifying the types of two expressions can be exponential [Mai90], but the pathological cases tend not to arise in practice.

CHECK YOUR UNDERSTANDING

13. What is the difference between *type equivalence* and *type compatibility*?
14. Discuss the comparative advantages of *structural* and *name* equivalence for types. Name three languages that use each approach.
15. Explain the difference between *strict* and *loose* name equivalence.
16. Explain the distinction between *derived* types and *subtypes* in Ada.
17. Explain the differences among *type conversion*, *type coercion*, and *nonconverting type casts*.
18. Summarize the arguments for and against coercion.
19. Under what circumstances does a type conversion require a run-time check?
20. What purpose is served by *universal* reference types?
21. What is *type inference*? Describe three contexts in which it occurs.
22. Under what circumstances does an ML compiler announce a type clash?
23. Explain how the type inference of ML leads naturally to polymorphism.
24. Why do ML programmers often declare the types of variables, even when they don't have to?
25. What is *unification*? What is its role in ML?

7.3 Parametric Polymorphism

As we have seen in the previous section, functions in ML-family languages are naturally polymorphic. Consider the simple task of finding the minimum of two values. In OCaml, the function

EXAMPLE 7.44
Finding the minimum in OCaml or Haskell

```
let min x y = if x < y then x else y;;
```

can be applied to arguments of any type, though sometimes the built-in definition of < may not be what the programmer would like. In Haskell the same function (minus the trailing semicolons) could be applied to arguments of any type in the class `Ord`; the programmer could add new types to this class by providing a definition of <. Sophisticated type inference allows the compiler to perform most checking at compile time in OCaml, and all of it in Haskell (see Sidebar 7.7 for details).

In OCaml, our `min` function would be said to have type `'a -> 'a -> 'a`; in Haskell, it would be `Ord a => a -> a -> a`. While the explicit parameters of `min` are `x` and `y`, we can think of `a` as an extra, implicit parameter—a *type parameter*. For this reason, ML-family languages are said to provide *implicit parametric polymorphism*. ■

Languages without compile-time type inference can provide similar convenience and expressiveness, if we are willing to delay type checking until run time. In Scheme, our `min` function would be written like this:

EXAMPLE 7.45
Implicit polymorphism in Scheme

```
(define min (lambda (a b) (if (< a b) a b)))
```

As in OCaml or Haskell, it makes no mention of types. The typical Scheme implementation employs an interpreter that examines the arguments to `min` and determines, at run time, whether they are mutually compatible and support a < operator. Given the definition above, the expression `(min 123 456)` evaluates to `123`; `(min 3.14159 2.71828)` evaluates to `2.71828`. The expression `(min "abc" "def")` produces a run-time error when evaluated, because the string comparison operator is named `string<?`, not <. ■

Similar run-time checks for object-oriented languages were pioneered by Smalltalk, and appear in Objective C, Swift, Python, and Ruby, among others. In these languages, an object is assumed to have an acceptable type if it supports whatever method is currently being invoked. In Ruby, for example, `min` is a predefined method supported by collection classes. Assuming that the elements of collection `C` support a comparison (`<=>` operator), `C.min` will return the minimum element:

EXAMPLE 7.46
Duck typing in Ruby

```
[5, 9, 3, 6].min                    # 3  (array)
(2..10).min                         # 2  (range)
["apple", "pear", "orange"].min     # "apple" (lexicographic order)
["apple", "pear", "orange"].min {
    |a,b| a.length <=> b.length
}                                   # "pear"
```

For the final call to `min`, we have provided, as a trailing block, an alternative definition of the comparison operator. ■

This operational style of checking (an object has an acceptable type if it supports the requested method) is sometimes known as *duck typing*. It takes its name from the notion that "if it walks like a duck and quacks like a duck, then it must be a duck." [6]

---

6 The origins of this "duck test" colloquialism are uncertain, but they go back at least as far as the early 20th century. Among other things, the test was widely cited in the 1940s and 50s as a means of identifying supposed Communist sympathizers.

7.3.1 Generic Subroutines and Classes

The disadvantage of polymorphism in Scheme, Smalltalk, Ruby, and the like is the need for run-time checking, which incurs nontrivial costs, and delays the reporting of errors. The implicit polymorphism of ML-family languages avoids these disadvantages, but requires advanced type inference. For other compiled languages, *explicit* parametric polymorphism (otherwise known as *generics*) allows the programmer to specify type parameters when declaring a subroutine or class. The compiler then uses these parameters in the course of static type checking.

EXAMPLE 7.47
Generic `min` function in Ada

Languages that provide generics include Ada, C++ (which calls them *templates*), Eiffel, Java, C#, and Scala. As a concrete example, consider the overloaded `min` functions on the left side of Figure 7.2. Here the integer and floating-point versions differ only in the types of the parameters and return value. We can exploit this similarity to define a single version that works not only for integers and reals, but for any type whose values are totally ordered. This code appears on the right side of Figure 7.2. The initial (bodyless) declaration of `min` is preceded by a `generic` clause specifying that two things are required in order to create a concrete instance of a minimum function: a type, `T`, and a corresponding comparison routine. This declaration is followed by the actual code for `min`, and instantiations of this code for integer and floating-point types. Given appropriate comparison routines (not shown), we can also instantiate versions for types like `string` and `date`, as shown on the last two lines. (The `"<"` operation mentioned in the definition of `string_min` is presumably overloaded; the compiler resolves the overloading by finding the version of `"<"` that takes arguments of type `T`, where `T` is already known to be `string`.) ■

EXAMPLE 7.48
Generic queues in C++

In an object-oriented language, generics are most often used to parameterize entire classes. Among other things, such classes may serve as *containers*—data abstractions whose instances hold a collection of other objects, but whose operations are generally oblivious to the type of the objects they contain. Examples of containers include stack, queue, heap, set, and dictionary (mapping) abstractions, implemented as lists, arrays, trees, or hash tables. In the absence of generics, it is possible in some languages (C is an obvious example, as were early versions of Java and C#) to define a queue of references to arbitrary objects, but use of such a queue requires type casts that abandon compile-time checking (Exercise 7.8). A simple generic queue in C++ appears in Figure 7.3. ■

EXAMPLE 7.49
Generic parameters

We can think of generic parameters as supporting compile-time customization, allowing the compiler to create an appropriate version of the parameterized subroutine or class. In some languages—Java and C#, for example—generic parameters must always be types. Other languages are more general. In Ada and C++, for example, a generic can be parameterized by values as well. We can see an example in Figure 7.3, where an integer parameter has been used to specify the

```
function min(x, y : integer)
    return integer is
begin
    if x < y then return x;
    else return y;
    end if;
end min;

function min(x, y : long_float)
    return long_float is
begin
    if x < y then return x;
    else return y;
    end if;
end min;
```

```
generic
    type T is private;
    with function "<"(x, y : T) return Boolean;
function min(x, y : T) return T;

function min(x, y : T) return T is
begin
    if x < y then return x;
    else return y;
    end if;
end min;

function int_min is new min(integer, "<");
function real_min is new min(long_float, "<");
function string_min is new min(string, "<");
function date_min is new min(date, date_precedes);
```

Figure 7.2 Overloading (*left*) versus generics (*right*) in Ada.

maximum length of the queue. In C++, this value must be a compile-time constant; in Ada, which supports dynamic-size arrays (Section 8.2.2), its evaluation can be delayed until elaboration time. ■

Implementation Options

Generics can be implemented several ways. In most implementations of Ada and C++ they are a purely static mechanism: all the work required to create and use multiple instances of the generic code takes place at compile time. In the usual case, the compiler creates a *separate copy* of the code for every instance. (C++

DESIGN & IMPLEMENTATION

7.9 Generics in ML

Perhaps surprisingly, given the implicit polymorphism that comes "for free" with type inference, both OCaml and SML provide explicit polymorphism—generics—as well, in the form of parameterized modules called *functors*. Unlike the implicit polymorphism, functors allow the OCaml or SML programmer to indicate that a collection of functions and other values (i.e., the contents of a module) share a common set of generic parameters. This sharing is then enforced by the compiler. Moreover, any types exported by a functor invocation (generic instantiation) are guaranteed to be distinct, even though their signatures (interfaces) are the same. As in Ada and C++, generic parameters in ML can be values as well as types.

NB: While Haskell also provides something called a `Functor` (specifically, a type class that supports a mapping function), its use of the term has little in common with that of OCaml and SML.

```
template<class item, int max_items = 100>
class queue {
    item items[max_items];
    int next_free, next_full, num_items;
public:
    queue() : next_free(0), next_full(0), num_items(0) { }
    bool enqueue(const item& it) {
        if (num_items == max_items) return false;
        ++num_items;  items[next_free] = it;
        next_free = (next_free + 1) % max_items;
        return true;
    }
    bool dequeue(item* it) {
        if (num_items == 0) return false;
        --num_items;  *it = items[next_full];
        next_full = (next_full + 1) % max_items;
        return true;
    }
};
...
queue<process> ready_list;
queue<int, 50> int_queue;
```

Figure 7.3 Generic array-based queue in C++.

goes farther, and arranges to type-check each of these instances independently.) If several queues are instantiated with the same set of arguments, then the compiler may share the code of the `enqueue` and `dequeue` routines among them. A clever compiler may arrange to share the code for a queue of integers with the code for a queue of floating-point numbers, if the two types happen to have the same size, but this sort of optimization is not required, and the programmer should not be surprised if it doesn't occur.

Java, by contrast, guarantees that *all* instances of a given generic will share the same code at run time. In effect, if `T` is a generic type parameter in Java, then objects of class `T` are treated as instances of the standard base class `Object`, except that the programmer does not have to insert explicit casts to use them as objects of class `T`, and the compiler guarantees, statically, that the elided casts will never fail. C# plots an intermediate course. Like C++, it will create specialized implementations of a generic for different primitive or value types. Like Java, however, it requires that the generic code itself be demonstrably type safe, independent of the arguments provided in any particular instantiation. We will examine the tradeoffs among C++, Java, and C# generics in more detail in Section C-7.3.2.

Generic Parameter Constraints

Because a generic is an abstraction, it is important that its interface (the header of its declaration) provide all the information that must be known by a user of the abstraction. Several languages, including Ada, Java, C#, Scala, OCaml, and SML,

attempt to enforce this rule by *constraining* generic parameters. Specifically, they require that the operations permitted on a generic parameter type be explicitly declared.

EXAMPLE 7.50
`with` constraints in Ada

In Ada, the programmer can specify the operations of a generic type parameter by means of a trailing `with` clause. We saw a simple example in the "minimum" function of Figure 7.2 (right side). The declaration of a generic sorting routine in Ada might be similar:

```
generic
    type T is private;
    type T_array is array (integer range <>) of T;
    with function "<"(a1, a2 : T) return boolean;
procedure sort(A : in out T_array);
```

Without the `with` clause, procedure `sort` would be unable to compare elements of `A` for ordering, because type `T` is `private`—it supports only assignment, testing for equality and inequality, and a few other standard attributes (e.g., `size`).

Java and C# employ a particularly clean approach to constraints that exploits the ability of object-oriented types to *inherit* methods from a parent type or interface. We defer a full discussion of inheritance to Chapter 10. For now, we note that it allows the Java or C# programmer to require that a generic parameter support a particular set of methods, much as the type classes of Haskell constrain the types of acceptable parameters to an implicitly polymorphic function. In Java, we might declare and use a sorting routine as follows:

EXAMPLE 7.51
Generic sorting routine in Java

DESIGN & IMPLEMENTATION

7.10 Overloading and polymorphism

Given that a compiler will often create multiple instances of the code for a generic subroutine, specialized to a given set of generic parameters, one might be forgiven for wondering: what exactly is the difference between the left and right sides of Figure 7.2? The answer lies in the generality of the polymorphic code. With overloading the programmer must write a separate `min` routine for every type, and while the compiler will choose among these automatically, the fact that they do something similar with their arguments is purely a matter of convention. Generics, on the other hand, allow the *compiler* to create an appropriate version for every needed type. The similarity of the calling syntax (and of the generated code, when conventions are followed) has led some authors to refer to overloading as *ad hoc* (special case) *polymorphism*. There is no particular reason, however, for the programmer to think of polymorphism in terms of multiple copies: from a semantic (conceptual) point of view, overloaded subroutines use a single name for more than one thing; a polymorphic subroutine *is* a single thing.

```
public static <T extends Comparable<T>> void sort(T A[]) {
    ...
    if (A[i].compareTo(A[j]) >= 0) ...
    ...
}
...
Integer[] myArray = new Integer[50];
...
sort(myArray);
```

Where C++ requires a `template<`*type_args*`>` prefix before a generic method, Java puts the type parameters immediately in front of the method's return type. The `extends` clause constitutes a generic constraint: `Comparable` is an interface (a set of required methods) from the Java standard library; it includes the method `compareTo`. This method returns −1, 0, or 1, respectively, depending on whether the current object is less than, equal to, or greater than the object passed as a parameter. The compiler checks to make sure that the objects in any array passed to `sort` are of a type that implements `Comparable`, and are therefore guaranteed to provide `compareTo`. If `T` had needed additional interfaces (that is, if we had wanted more constraints), they could have been specified with a comma-separated list: `<T extends I1, I2, I3>`. ■

EXAMPLE 7.52
Generic sorting routine in C#

C# syntax is similar:

```
static void sort<T>(T[] A) where T : IComparable {
    ...
    if (A[i].CompareTo(A[j]) >= 0) ...
    ...
}
...
int[] myArray = new int[50];
sort(myArray);
```

C# puts the type parameters after the name of the subroutine, and the constraints (the `where` clause) after the regular parameter list. The compiler is smart enough to recognize that `int` is a primitive type, and generates a customized implementation of `sort`, eliminating the need for Java's `Integer` wrapper class, and producing faster code. ■

EXAMPLE 7.53
Generic sorting routine in C++

A few languages forgo explicit constraints, but still check how parameters are used. In C++, for example, the header of a generic sorting routine can be extremely simple:

```
template<typename T>
void sort(T A[], int A_size) { ...
```

No mention is made of the need for a comparison operator. The body of a generic can (attempt to) perform arbitrary operations on objects of a generic parameter

type, but if the generic is instantiated with a type that does not support that operation, the compiler will announce a static semantic error. Unfortunately, because the header of the generic does not necessarily specify which operations will be required, it can be difficult for the programmer to predict whether a particular instantiation will cause an error message. Worse, in some cases the type provided in a particular instantiation may support an operation required by the generic's code, but that operation may not do "the right thing." Suppose in our C++ sorting example that the code for `sort` makes use of the `<` operator. For `ints` and `doubles`, this operator will do what one would expect. For character strings, however, it will compare pointers, to see which referenced character has a lower address in memory. If the programmer is expecting comparison for lexicographic ordering, the results may be surprising!

To avoid surprises, it is best to avoid implicit use of the operations of a generic parameter type. The next version of the C++ standard is likely to incorporate syntax for explicit template constraints [SSD13]. For now, the comparison routine can be provided as a method of class `T`, an extra argument to the `sort` routine, or an extra generic parameter. To facilitate the first of these options, the programmer may choose to emulate Java or C#, encapsulating the required methods in an abstract base class from which the type `T` may inherit. ■

Implicit Instantiation

EXAMPLE 7.54
Generic class instance in C++

Because a class is a type, one must generally create an instance of a generic class (i.e., an object) before the generic can be used. The declaration provides a natural place to provide generic arguments:

```
queue<int, 50> *my_queue = new queue<int, 50>();      // C++
```
■

EXAMPLE 7.55
Generic subroutine instance in Ada

Some languages (Ada among them) also require generic subroutines to be instantiated explicitly before they can be used:

```
procedure int_sort is new sort(integer, int_array, "<");
...
int_sort(my_array);
```
■

EXAMPLE 7.56
Implicit instantiation in C++

Other languages (C++, Java, and C# among them) do not require this. Instead they treat generic subroutines as a form of overloading. Given the C++ sorting routine of Example 7.53 and the following objects:

```
int ints[10];
double reals[50];
string strings[30];  // library class string has lexicographic operator<
```

we can perform the following calls without instantiating anything explicitly:

```
sort(ints, 10);
sort(reals, 50);
sort(strings, 30);
```

	Explicit (generics)				Implicit	
	Ada	**C++**	**Java**	**C#**	**Lisp**	**ML**
Applicable to	subroutines, modules	subroutines, classes	subroutines, classes	subroutines, classes	functions	functions
Abstract over	types; subroutines; values of arbitrary types	types; enum, int, and pointer constants	types only	types only	types only	types only
Constraints	explicit (varied)	implicit	explicit (inheritance)	explicit (inheritance)	implicit	implicit
Checked at	compile time (definition)	compile time (instantiation)	compile time (definition)	compile time (definition)	run time	compile time (inferred)
Natural implementation	multiple copies	multiple copies	single copy (erasure)	multiple copies (reification)	single copy	single copy
Subroutine instantiation	explicit	implicit	implicit	implicit	—	—

Figure 7.4 Mechanisms for parametric polymorphism in Ada, C++, Java, C#, Lisp, and ML. Erasure and reification are discussed in Section C-7.3.2.

In each case, the compiler will implicitly instantiate an appropriate version of the `sort` routine. Java and C# have similar conventions. To keep the language manageable, the rules for implicit instantiation in C++ are more restrictive than the rules for resolving overloaded subroutines in general. In particular, the compiler will not coerce a subroutine argument to match a type expression containing a generic parameter (Exercise C-7.26). ■

Figure 7.4 summarizes the features of Ada, C++, Java, and C# generics, and of the implicit parametric polymorphism of Lisp and ML. Further explanation of some of the details appears in Section C-7.3.2.

7.3.2 Generics in C++, Java, and C#

Several of the key tradeoffs in the design of generics can be illustrated by comparing the features of C++, Java, and C#. C++ is by far the most ambitious of the three. Its templates are intended for almost any programming task that requires substantially similar but not identical copies of an abstraction. Java and C# provide generics purely for the sake of polymorphism. Java's design was heavily influenced by the desire for backward compatibility, not only with existing versions of the language, but with existing virtual machines and libraries. The C# designers, though building on an existing language, did not feel as constrained. They had been planning for generics from the outset, and were able to engineer substantial new support into the .NET virtual machine.

IN MORE DEPTH

On the companion site we discuss C++, Java, and C# generics in more detail, and consider the impact of their differing designs on the quality of error messages, the speed and size of generated code, and the expressive power of the notation. We note in particular the very different mechanisms used to make generic classes and methods support as broad a class of generic arguments as possible.

7.4 Equality Testing and Assignment

For simple, primitive data types such as integers, floating-point numbers, or characters, equality testing and assignment are relatively straightforward operations, with obvious semantics and obvious implementations (bit-wise comparison or copy). For more complicated or abstract data types, both semantic and implementation subtleties arise.

Consider for example the problem of comparing two character strings. Should the expression `s = t` determine whether `s` and `t`

- are aliases for one another?
- occupy storage that is bit-wise identical over its full length?
- contain the same sequence of characters?
- would appear the same if printed?

The second of these tests is probably too low level to be of interest in most programs; it suggests the possibility that a comparison might fail because of garbage in currently unused portions of the space reserved for a string. The other three alternatives may all be of interest in certain circumstances, and may generate different results.

In many cases the definition of equality boils down to the distinction between l-values and r-values: in the presence of references, should expressions be considered equal only if they refer to the same object, or also if the objects to which they refer are in some sense equal? The first option (refer to the same object) is known as a *shallow* comparison. The second (refer to equal objects) is called a *deep* comparison. For complicated data structures (e.g., lists or graphs) a deep comparison may require recursive traversal.

In imperative programming languages, assignment operations may also be deep or shallow. Under a reference model of variables, a shallow assignment `a := b` will make `a` refer to the object to which `b` refers. A deep assignment will create a copy of the object to which `b` refers, and make `a` refer to the copy. Under a value model of variables, a shallow assignment will copy the value of `b` into `a`, but if that value is a pointer (or a record containing pointers), then the objects to which the pointer(s) refer will not be copied.

Most programming languages employ both shallow comparisons and shallow assignment. A few (notably Python and the various dialects of Lisp and ML)

EXAMPLE 7.57
Equality testing in Scheme

provide more than one option for comparison. Scheme, for example, has three general-purpose equality-testing functions:

```
(eq? a b)        ; do a and b refer to the same object?
(eqv? a b)       ; are a and b known to be semantically equivalent?
(equal? a b)     ; do a and b have the same recursive structure?
```

Both `eq?` and `eqv?` perform a shallow comparison. The former may be faster for certain types in certain implementations; in particular, `eqv?` is required to detect the equality of values of the same discrete type, stored in different locations; `eq?` is not. The simpler `eq?` behaves as one would expect for Booleans, symbols (names), and pairs (things built by `cons`), but can have implementation-defined behavior on numbers, characters, and strings:

```
(eq? #t #t)              ⟹  #t (true)
(eq? 'foo 'foo)          ⟹  #t
(eq? '(a b) '(a b))      ⟹  #f (false); created by separate cons-es
(let ((p '(a b)))
  (eq? p p))             ⟹  #t; created by the same cons
(eq? 2 2)                ⟹  implementation dependent
(eq? "foo" "foo")        ⟹  implementation dependent
```

In any particular implementation, numeric, character, and string tests will always work the same way; if `(eq? 2 2)` returns `true`, then `(eq? 37 37)` will return `true` also. Implementations are free to choose whichever behavior results in the fastest code.

The exact rules that govern the situations in which `eqv?` is guaranteed to return `true` or `false` are quite involved. Among other things, they specify that `eqv?` should behave as one might expect for numbers, characters, and nonempty strings, and that two objects will never test `true` for `eqv?` if there are any circumstances under which they would behave differently. (Conversely, however, `eqv?` is allowed to return `false` for certain objects—functions, for example—that would behave identically in all circumstances.)[7] The `eqv?` predicate is "less discriminating" than `eq?`, in the sense that `eqv?` will never return `false` when `eq?` returns `true`.

For structures (lists), `eqv?` returns `false` if its arguments refer to different root `cons` cells. In many programs this is not the desired behavior. The `equal?` predicate recursively traverses two lists to see if their internal structure is the same and their leaves are `eqv?`. The `equal?` predicate may lead to an infinite loop if the programmer has used the imperative features of Scheme to create a circular list. ■

---

7 Significantly, `eqv?` is also allowed to return false when comparing numeric values of different types: `(eqv? 1 1.0)` may evaluate to `#f`. For numeric code, one generally wants the separate `=` function: `(= val1 val2)` will perform the necessary coercion and test for numeric equality (subject to rounding errors).

Deep assignments are relatively rare. They are used primarily in distributed computing, and in particular for parameter passing in remote procedure call (RPC) systems. These will be discussed in Section C-13.5.4.

For user-defined abstractions, no single language-specified mechanism for equality testing or assignment is likely to produce the desired results in all cases. Languages with sophisticated data abstraction mechanisms usually allow the programmer to define the comparison and assignment operators for each new data type—or to specify that equality testing and/or assignment is not allowed.

✓ CHECK YOUR UNDERSTANDING

26. Explain the distinction between implicit and explicit parametric polymorphism. What are their comparative advantages?
27. What is *duck typing*? What is its connection to polymorphism? In what languages does it appear?
28. Explain the distinction between overloading and generics. Why is the former sometimes called *ad hoc* polymorphism?
29. What is the principal purpose of generics? In what sense do generics serve a broader purpose in C++ and Ada than they do in Java and C#?
30. Under what circumstances can a language implementation share code among separate instances of a generic?
31. What are *container* classes? What do they have to do with generics?
32. What does it mean for a generic parameter to be *constrained*? Explain the difference between explicit and implicit constraints. Describe how interface classes can be used to specify constraints in Java and C#.
33. Why will C# accept `int` as a generic argument, but Java won't?
34. Under what circumstances will C++ instantiate a generic function implicitly?
35. Why is equality testing more subtle than it first appears?

7.5 Summary and Concluding Remarks

This chapter has surveyed the fundamental concept of *types*. In the typical programming language, types serve two principal purposes: they provide implicit context for many operations, freeing the programmer from the need to specify that context explicitly, and they allow the compiler to catch a wide variety of common programming errors. When discussing types, we noted that it is sometimes helpful to distinguish among denotational, structural, and abstraction-based points of view, which regard types, respectively, in terms of their values, their substructure, and the operations they support.

In a typical programming language, the *type system* consists of a set of built-in types, a mechanism to define new types, and rules for *type equivalence*, *type compatibility*, and *type inference*. Type equivalence determines when two values or named objects have the same type. Type compatibility determines when a value of one type may be used in a context that "expects" another type. Type inference determines the type of an expression based on the types of its components or (sometimes) the surrounding context. A language is said to be *strongly typed* if it never allows an operation to be applied to an object that does not support it; a language is said to be *statically typed* if it enforces strong typing at compile time.

We introduced terminology for the common built-in types and for enumerations, subranges, and the common type *constructors* (more on the latter will appear in Chapter 8). We discussed several different approaches to type equivalence, compatibility, and inference. We also examined type *conversion*, *coercion*, and *nonconverting casts*. In the area of type equivalence, we contrasted the *structural* and *name*-based approaches, noting that while name equivalence appears to have gained in popularity, structural equivalence retains its advocates.

Expanding on material introduced in Section 3.5.2, we explored several styles of *polymorphism*, all of which allow a subroutine—or the methods of a class—to operate on values of multiple types, so long as they only use those values in ways their types support. We focused in particular on *parametric* polymorphism, in which the types of the values on which the code will operate are passed to it as extra parameters, implicitly or explicitly. The implicit alternative appears in the static typing of ML and its descendants, and in the dynamic typing of Lisp, Smalltalk, and many other languages. The explicit alternative appears in the *generics* of many modern languages. In Chapter 10 we will consider the related topic of *subtype polymorphism*.

In our discussion of implicit parametric polymorphism, we devoted considerable attention to type checking in ML, where the compiler uses a sophisticated system of inference to determine, at compile time, whether a type error (an attempt to perform an operation on a type that doesn't support it) could ever occur at run time—all without access to type declarations in the source code. In our discussion of generics we explored alternative ways to express *constraints* on generic parameters. We also considered implementation strategies. As examples, we contrasted (on the companion site) the generic facilities of C++, Java, and C#.

More so, perhaps, than in previous chapters, our study of types has highlighted fundamental differences in philosophy among language designers. As we have seen, some languages use variables to name values; others, references. Some languages do all or most of their type checking at compile time; others wait until run time. Among those that check at compile time, some use name equivalence; others, structural equivalence. Some languages avoid type coercions; others embrace them. Some avoid overloading; others again embrace them. In each case, the choice among design alternatives reflects nontrivial tradeoffs among competing language goals, including expressiveness, ease of programming, quality and timing of error discovery, ease of debugging and maintenance, compilation cost, and run-time performance.

7.6 Exercises

7.1 Most statically typed languages developed since the 1970s (including Java, C#, and the descendants of Pascal) use some form of name equivalence for types. Is structural equivalence a bad idea? Why or why not?

7.2 In the following code, which of the variables will a compiler consider to have compatible types under structural equivalence? Under strict name equivalence? Under loose name equivalence?

```
type  T = array [1..10] of integer
      S = T
A  : T
B  : T
C  : S
D  : array [1..10] of integer
```

7.3 Consider the following declarations:

```
1.  type cell              -- a forward declaration
2.  type cell_ptr = pointer to cell
3.  x : cell
4.  type cell = record
5.        val : integer
6.        next : cell_ptr
7.  y : cell
```

Should the declaration at line 4 be said to introduce an alias type? Under strict name equivalence, should x and y have the same type? Explain.

7.4 Suppose you are implementing an Ada compiler, and must support arithmetic on 32-bit fixed-point binary numbers with a programmer-specified number of fractional bits. Describe the code you would need to generate to add, subtract, multiply, or divide two fixed-point numbers. You should assume that the hardware provides arithmetic instructions only for integers and IEEE floating-point. You may assume that the integer instructions preserve full precision; in particular, integer multiplication produces a 64-bit result. Your description should be general enough to deal with operands and results that have different numbers of fractional bits.

7.5 When Sun Microsystems ported Berkeley Unix from the Digital VAX to the Motorola 680x0 in the early 1980s, many C programs stopped working, and had to be repaired. In effect, the 680x0 revealed certain classes of program bugs that one could "get away with" on the VAX. One of these classes of bugs occurred in programs that use more than one size of integer (e.g., `short` and `long`), and arose from the fact that the VAX is a little-endian machine, while the 680x0 is big-endian (Section C-5.2). Another class of bugs occurred in programs that manipulate both null and empty strings. It arose

from the fact that location zero in a Unix process's address space on the VAX always contained a zero, while the same location on the 680x0 is not in the address space, and will generate a protection error if used. For both of these classes of bugs, give examples of program fragments that would work on a VAX but not on a 680x0.

7.6 Ada provides two "remainder" operators, `rem` and `mod` for integer types, defined as follows [Ame83, Sec. 4.5.5]:

> Integer division and remainder are defined by the relation `A = (A/B)*B + (A rem B)`, where `(A rem B)` has the sign of `A` and an absolute value less than the absolute value of `B`. Integer division satisfies the identity `(-A)/B = -(A/B) = A/(-B)`.
>
> The result of the modulus operation is such that `(A mod B)` has the sign of `B` and an absolute value less than the absolute value of `B`; in addition, for some integer value `N`, this result must satisfy the relation `A = B*N + (A mod B)`.

Give values of `A` and `B` for which `A rem B` and `A mod B` differ. For what purposes would one operation be more useful than the other? Does it make sense to provide both, or is it overkill?

Consider also the `%` operator of C and the `mod` operator of Pascal. The designers of these languages could have picked semantics resembling those of either Ada's `rem` or its `mod`. Which did they pick? Do you think they made the right choice?

7.7 Consider the problem of performing range checks on set expressions in Pascal. Given that a set may contain many elements, some of which may be known at compile time, describe the information that a compiler might maintain in order to track both the elements known to belong to the set and the possible range of unknown elements. Then explain how to update this information for the following set operations: union, intersection, and difference. The goal is to determine (1) when subrange checks can be eliminated at run time and (2) when subrange errors can be reported at compile time. Bear in mind that the compiler *cannot* do a perfect job: some unnecessary run-time checks will inevitably be performed, and some operations that must always result in errors will not be caught at compile time. The goal is to do as good a job as possible at reasonable cost.

7.8 In Section 7.2.2 we introduced the notion of a *universal reference type* (`void *` in C) that refers to an object of unknown type. Using such references, implement a "poor man's generic queue" in C, as suggested in Section 7.3.1. Where do you need type casts? Why? Give an example of a use of the queue that will fail catastrophically at run time, due to the lack of type checking.

7.9 Rewrite the code of Figure 7.3 in Ada, Java, or C#.

7.10 (a) Give a generic solution to Exercise 6.19.

(b) Translate this solution into Ada, Java, or C#.

7.11 In your favorite language with generics, write code for simple versions of the following abstractions:

(a) a stack, implemented as a linked list

(b) a priority queue, implemented as a skip list or a partially ordered tree embedded in an array

(c) a dictionary (mapping), implemented as a hash table

7.12 Figure 7.3 passes integer `max_items` to the `queue` abstraction as a generic parameter. Write an alternative version of the code that makes `max_items` a parameter to the `queue` constructor instead. What is the advantage of the generic parameter version?

7.13 Rewrite the generic sorting routine of Examples 7.50–7.52 (with constraints) using OCaml or SML functors.

7.14 Flesh out the C++ sorting routine of Example 7.53. Demonstrate that this routine does "the wrong thing" when asked to sort an array of `char*` strings.

7.15 In Example 7.53 we mentioned three ways to make the need for comparisons more explicit when defining a generic sort routine in C++: make the comparison routine a method of the generic parameter class `T`, an extra argument to the sort routine, or an extra generic parameter. Implement these options and discuss their comparative strengths and weaknesses.

7.16 Yet another solution to the problem of the previous exercise is to make the sorting routine a method of a `sorter` class. The comparison routine can then be passed into the class as a constructor argument. Implement this option and compare it to those of the previous exercise.

7.17 Consider the following code skeleton in C++:

```
#include <list>
using std::list;

class foo { ...
class bar : public foo { ...

static void print_all(list<foo*> &L) { ...

list<foo*> LF;
list<bar*> LB;
...
print_all(LF);        // works fine
print_all(LB);        // static semantic error
```

Explain why the compiler won't allow the second call. Give an example of bad things that could happen if it did.

7.18 Bjarne Stroustrup, the original designer of C++, once described templates as "a clever kind of macro that obeys the scope, naming, and type rules of C++" [Str13, 2nd ed., p. 257]. How close is the similarity? What can templates do that macros can't? What do macros do that templates don't?

7.19 In Section 9.3.1 we noted that Ada 83 does not permit subroutines to be passed as parameters, but that some of the same effect can be achieved with generics. Suppose we want to apply a function to every member of an array. We might write the following in Ada 83:

```
generic
    type item is private;
    type item_array is array (integer range <>) of item;
    with function F(it : in item) return item;
procedure apply_to_array(A : in out item_array);

procedure apply_to_array(A : in out item_array) is
begin
    for i in A'first..A'last loop
        A(i) := F(A(i));
    end loop;
end apply_to_array;
```

Given an array of integers, `scores`, and a function on integers, `foo`, we can write:

```
procedure apply_to_ints is
    new apply_to_array(integer, int_array, foo);
...
apply_to_ints(scores);
```

How general is this mechanism? What are its limitations? Is it a reasonable substitute for formal (i.e., second-class, as opposed to third-class) subroutines?

7.20 Modify the code of Figure 7.3 or your solution to Exercise 7.12 to throw an exception if an attempt is made to enqueue an item in a full queue, or to dequeue an item from an empty queue.

7.21–7.27 In More Depth.

7.7 Explorations

7.28 Some language definitions specify a particular representation for data types in memory, while others specify only the semantic behavior of those types. For languages in the latter class, some implementations guarantee a particular representation, while others reserve the right to choose different representations in different circumstances. Which approach do you prefer? Why?

7.29 Investigate the *typestate* mechanism employed by Strom et al. in the Hermes programming language [SBG+91]. Discuss its relationship to the notion of definite assignment in Java and C# (Section 6.1.3).

7.30 Several recent projects attempt to blur the line between static and dynamic typing by adding optional type declarations to scripting languages. These declarations support a strategy of *gradual typing*, in which programmers initially write in a traditional scripting style and then add declarations incrementally to increase reliability or decrease run-time cost. Learn about the Dart, Hack, and TypeScript languages, promoted by Google, Facebook, and Microsoft, respectively. What are your impressions? How easy do you think it will be in practice to retrofit declarations into programs originally developed without them?

7.31 Research the type systems of Standard ML, OCaml, Haskell, and F#. What are the principal differences? What might explain the different choices made by the language designers?

7.32 Write a program in C++ or Ada that creates at least two concrete types or subroutines from the same template/generic. Compile your code to assembly language and look at the result. Describe the mapping from source to target code.

7.33 While Haskell does not include generics (its parametric polymorphism is implicit), its type classes can be considered a generalization of type constraints. Learn more about type classes. Discuss their relevance to polymorphic functions, as well as more general uses. You might want to look ahead to the discussion of *monads* in Section 11.5.2.

7.34 Investigate the notion of *type conformance*, employed by Black et al. in the Emerald programming language [BHJL07]. Discuss how conformance relates to the type inference of ML and to the class-based typing of object-oriented languages.

7.35 C++11 introduces so-called *variadic templates*, which take a variable number of generic parameters. Read up on how these work. Show how they might be used to replace the usual `cout` << $expr_1$ << ... << $expr_n$ syntax of formatted output with `print(`$expr_1$`, ..., `$expr_n$`)`, while retaining full static type checking.

7.36–7.38 In More Depth.

7.8 Bibliographic Notes

References to general information on the various programming languages mentioned in this chapter can be found in Appendix A, and in the Bibliographic Notes for Chapters 1 and 6. Welsh, Sneeringer, and Hoare [WSH77] provide a critique of the original Pascal definition, with a particular emphasis on its type system. Tanenbaum's comparison of Pascal and Algol 68 also focuses largely on types [Tan78]. Cleaveland [Cle86] provides a book-length study of many of the issues in this chapter. Pierce [Pie02] provides a formal and detailed modern coverage of the subject. The ACM Special Interest Group on Programming Languages

launched a biennial workshop on *Types in Language Design and Implementation* in 2003.

What we have referred to as the denotational model of types originates with Hoare [DDH72]. Denotational formulations of the overall semantics of programming languages are discussed in the Bibliographic Notes for Chapter 4. A related but distinct body of work uses algebraic techniques to formalize data abstraction; key references include Guttag [Gut77] and Goguen et al. [GTW78]. Milner's original paper [Mil78] is the seminal reference on type inference in ML. Mairson [Mai90] proves that the cost of unifying ML types is $O(2^n)$, where n is the length of the program. Fortunately, the cost is linear in the size of the program's type expressions, so the worst case arises only in programs whose semantics are too complex for a human being to understand anyway.

Hoare [Hoa75] discusses the definition of recursive types under a reference model of variables. Cardelli and Wegner survey issues related to polymorphism, overloading, and abstraction [CW85]. The Character Model standard for the World Wide Web provides a remarkably readable introduction to the subtleties and complexities of multilingual character sets [Wor05].

Garcia et al. provide a detailed comparison of generic facilities in ML, C++, Haskell, Eiffel, Java, and C# [GJL+03]. The C# generic facility is described by Kennedy and Syme [KS01]. Java generics are based on the work of Bracha et al. [BOSW98]. Erwin Unruh is credited with discovering that C++ templates could trick the compiler into performing nontrivial computation. His specific example (*www.erwin-unruh.de/primorig.html*) did not compile, but caused the compiler to generate a sequence of n error messages, embedding the first n primes. Abrahams and Gurtovoy provide a book-length treatment of *template metaprogramming* [AG05], the field that grew out of this discovery.

This page intentionally left blank